

Manual de Python

Do primeiro script ao código de produção — a linguagem, a biblioteca padrão e o ecossistema

Vitor Mattos

10/08/2026

Índice

Apresentação	1
Para quem é este manual	1
Como o manual está organizado	2
Seções fixas de cada capítulo	3
Sobre o código deste livro	3
Licença	3
I. Volume 1 — Primeiros Passos com Python	5
1. O que é Python: história, filosofia e o Zen da linguagem	7
1.1. Seis linhas e vinte e quatro	7
1.2. De um projeto de fim de ano a uma linguagem de infraestrutura	8
1.3. A ruptura do Python 3 e a lição que ficou	9
1.4. Quem decide o que entra na linguagem	9
1.5. Python 3.13, a versão de referência deste manual	10
1.6. O Zen do Python	11
1.7. Onde Python é forte e onde não é	13
1.8. Jeito Pythonico	13
1.9. Laboratório	14
1.10. Resumo	17
2. Como o Python executa código: interpretador, bytecode e CPython	19
2.1. O erro que chega antes da primeira linha	19
2.2. As quatro etapas	20
2.3. O bytecode por dentro	21
2.4. <code>__pycache__</code> , o cache que aparece sozinho	24
2.5. CPython e as outras implementações	25
2.6. Jeito Pythonico	26
2.7. Laboratório	27
2.8. Resumo	29
3. Instalando o Python: versões, gerenciadores e o Python do sistema	31
3.1. O Python do sistema não é seu	31
3.2. As três camadas	33
3.3. O ambiente virtual, com precisão	33
3.4. Quando você precisa de outra versão	36
3.5. Windows e macOS em duas páginas	37
3.6. Jeito Pythonico	37
3.7. Laboratório	38
3.8. Resumo	41
4. O REPL, os scripts e o primeiro programa	43
4.1. O REPL: onde se descobre	43
4.2. O script: onde se repete	46

4.3. <code>if __name__ == "__main__"</code>	47
4.4. As cinco formas de rodar código Python	49
4.5. Jeito Pythônico	50
4.6. Laboratório	51
4.7. Resumo	54
5. Ambiente de trabalho: editor, VS Code, Jupyter e o terminal	55
5.1. Uma regra que resolve a maioria dos problemas	55
5.2. O terminal	56
5.3. O editor	56
5.4. Linter e formatador: <code>ruff</code>	57
5.5. O notebook	60
5.6. Outras opções	62
5.7. Jeito Pythônico	62
5.8. Laboratório	63
5.9. Resumo	66
6. Sintaxe essencial: indentação, linhas lógicas e comentários	67
6.1. O bloco é a indentação	67
6.2. Linha física e linha lógica	69
6.3. Comando e expressão	71
6.4. Nomes	72
6.5. Comentários e documentação	73
6.6. Uma nota sobre codificação	74
6.7. Jeito Pythônico	74
6.8. Laboratório	75
6.9. Resumo	78
7. Erros e tracebacks: como ler o que o Python está dizendo	81
7.1. O traceback do começo ao fim	81
7.2. As exceções que você vai ver mais	83
7.3. Exceções encadeadas	85
7.4. Quando o traceback é gigante	87
7.5. <code>SyntaxError</code> é diferente de todos os outros	87
7.6. Jeito Pythônico	88
7.7. Laboratório	90
7.8. Resumo	93
II. Volume 2 — Tipos, Operadores e Expressões	95
8. Objetos, variáveis e o modelo de referências	97
9. Números inteiros, de ponto flutuante e complexos	99
10. Precisão numérica: decimal, fractions e as armadilhas do float	101
11. Booleanos, valor-verdade e os operadores lógicos	103
12. Strings: criação, indexação e fatiamento	105
13. Métodos de string, formatação e f-strings	107

14. Unicode, bytes e codificação de caracteres	109
III. Volume 3 — Controle de Fluxo e Funções	111
15. Condicionais: if, elif, else e a expressão condicional	113
16. Laços: while, for e o protocolo de iteração	115
17. break, continue, else em laços e o casamento de padrões	117
18. Definindo funções: parâmetros, argumentos e retorno	119
19. Argumentos nomeados, valores padrão, *args e **kwargs	121
20. Escopo, closures e a regra LEGB	123
21. Funções como objetos: lambda, map, filter e ordem superior	125
IV. Volume 4 — Estruturas de Dados Nativas	127
22. Listas: operações, mutabilidade e cópia	129
23. Tuplas, empacotamento e desempacotamento	131
24. Dicionários: chaves, ordem e os métodos essenciais	133
25. Conjuntos e operações de conjunto	135
26. Compreensões de lista, dicionário e conjunto	137
27. Ordenação, chaves de ordenação e comparação de objetos	139
28. Escolhendo a estrutura certa: custo das operações	141
V. Volume 5 — Módulos, Pacotes e o Ambiente	143
29. Módulos, import e o caminho de busca	145
30. Pacotes, __init__.py e a organização de um projeto	147
31. Ambientes virtuais: venv, pip e o isolamento de dependências	149
32. Empacotamento moderno: pyproject.toml, build e publicação	151
33. uv, pipx e Poetry: o ecossistema de gerenciamento	153
34. Ponto de entrada: __main__, scripts de console e a linha de comando	155
VI. Volume 6 — Arquivos, Erros e Contextos	157
35. Leitura e escrita de arquivos de texto	159

36.Arquivos binários, buffers e o módulo io	161
37.Caminhos e o sistema de arquivos com pathlib	163
38.Exceções: hierarquia e o bloco try, except, else e finally	165
39.Levantando exceções e criando exceções próprias	167
40.Gerenciadores de contexto e a instrução with	169
VII.Volume 7 — Programação Orientada a Objetos	171
41.Classes, instâncias e o self	173
42.Atributos, propriedades e encapsulamento	175
43.Herança, MRO e a função super	177
44.Composição, delegação e quando não herdar	179
45.Métodos estáticos, métodos de classe e o desenho da API	181
46.Dataclasses, NamedTuple e enumerações	183
47.Classes abstratas, protocolos e duck typing	185
VIII.Volume 8 — O Modelo de Dados de Python	187
48.Métodos especiais: o protocolo por trás da sintaxe	189
49.Iteradores e o protocolo de iteração	191
50.Geradores, yield e a avaliação preguiçosa	193
51.Decoradores de função	195
52.Decoradores de classe, functools e metadados preservados	197
53.Descriptores, __slots__ e o acesso a atributos	199
54.Metaclasses e __init_subclass__: quando (não) usar	201
IX. Volume 9 — A Biblioteca Padrão Essencial	203
55.os, sys e a interação com o sistema	205
56.datetime, zoneinfo e o tratamento de tempo	207
57.collections: deque, Counter, defaultdict e namedtuple	209
58.itertools e functools: a caixa de ferramentas funcional	211
59.math, statistics e random	213

60.logging: registrando o que o programa faz	215
X. Volume 10 — Texto, Dados e Serialização	217
61.Expressões regulares com o módulo re	219
62.JSON, CSV e formatos tabulares	221
63.YAML, TOML e arquivos de configuração	223
64.Serialização binária: pickle, struct e os riscos envolvidos	225
65.SQLite e a DB-API: banco de dados sem servidor	227
66.hashlib, secrets e o tratamento de dados sensíveis	229
XI. Volume 11 — Qualidade: Testes, Tipagem e Ferramentas	231
67.Estilo, PEP 8 e formatação automática com Ruff e Black	233
68.Anotações de tipo: fundamentos e o módulo typing	235
69.Tipagem avançada: genéricos, protocolos e mypy	237
70.Testes com pytest: fundamentos, asserções e fixtures	239
71.Parametrização, dublês de teste e cobertura	241
72.Depuração: pdb, breakpoint e o traceback difícil	243
73.Documentação: docstrings, doctest e Sphinx	245
XII. Volume 12 — Automação, Sistema e Redes	247
74.Scripts de automação: arquivos, planilhas e tarefas repetitivas	249
75.subprocess: chamando programas externos com segurança	251
76.Interfaces de linha de comando com argparse, Click e Typer	253
77.Redes com sockets: TCP, UDP e o modelo cliente-servidor	255
78.Requisições HTTP com requests e httpx	257
79.Raspagem de dados responsável: HTML, seletores e limites éticos	259
XIII. Volume 13 — Dados, Análise e Visualização	261
80.NumPy: arrays, vetorização e broadcasting	263
81.pandas: Series, DataFrame e a carga de dados	265

82. Limpeza, transformação e agregação com pandas	267
83. Visualização de dados com Matplotlib	269
84. SciPy e a computação científica aplicada	271
85. Introdução ao aprendizado de máquina com scikit-learn	273
XIV. Volume 14 — Web, APIs e Bancos de Dados	275
86. Como funciona uma aplicação web: HTTP, WSGI e ASGI	277
87. Validação de dados com Pydantic	279
88. APIs REST com FastAPI	281
89. Flask e Django: quando usar cada um	283
90. Bancos de dados relacionais com SQLAlchemy	285
91. Autenticação, autorização e segurança em aplicações Python	287
92. Implantação: contêineres, servidores de aplicação e produção	289
XV. Volume 15 — Concorrência, Desempenho e Internos	291
93. Concorrência, paralelismo e o GIL	293
94. Threads e o módulo threading	295
95. Multiprocessing e o paralelismo real	297
96. Programação assíncrona: async, await e asyncio	299
97. Medindo desempenho: timeit, profiling e otimização	301
98. Memória, contagem de referências e o coletor de lixo	303
XVI. Volume 16 — Fronteira e Prática Contínua	305
99. Estendendo Python: C, Cython e ligações nativas	307
100. Python acelerado: PyPy, Numba e o interpretador sem GIL	309
101. Arquitetura de projetos e código sustentável	311
102. Git, pre-commit e integração contínua para projetos Python	313
103. Python e inteligência artificial: LLMs, APIs e agentes	315
104. Para onde ir depois: PEPs, comunidade e prática contínua	317
Referências	319

Apresentação

Este é um manual de **Python** escrito em português do Brasil, pensado para levar o leitor do primeiro `print` até o código que roda em produção — passando pela linguagem a fundo, pela biblioteca padrão, pelas ferramentas que separam script de software e pelos domínios em que Python se tornou padrão de fato.

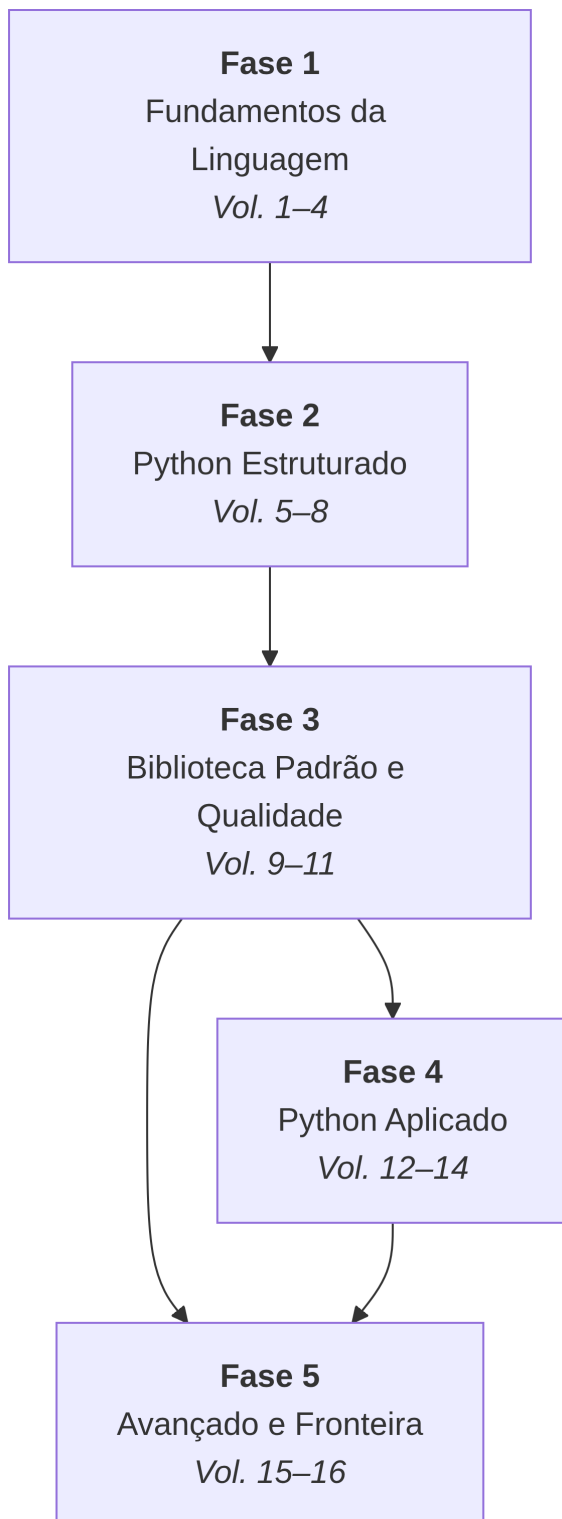
A versão de referência é o **Python 3.13**. O código é moderno: `pathlib` no lugar de `os.path`, f-strings no lugar do operador `%`, `list[int]` no lugar de `typing.List[int]`. Onde um recurso é recente, o texto diz a partir de qual versão ele existe; onde o código antigo ainda é comum em produção, explica o que se fazia antes e por que mudou. O ambiente de referência é Linux, com notas para Windows e macOS onde a diferença importa.

Para quem é este manual

Para quem quer aprender Python a sério e não parar no básico. Serve a quem nunca programou — o Volume 1 começa do zero, sem pressupor outra linguagem —, a quem já programa em outra linguagem e quer entender o que é específico de Python em vez de escrever Java com sintaxe de Python, e a quem já usa Python há anos de forma intuitiva e quer fechar as lacunas: descritores, o modelo de dados, o GIL, empacotamento, tipagem.

Não é preciso conhecimento prévio de programação. É preciso, sim, um interpretador aberto ao lado enquanto se lê.

Como o manual está organizado



Dependências entre as fases do manual

São **104 capítulos** distribuídos em **16 volumes** e **5 fases**, escritos para serem lidos em sequência por quem está começando, e consultados por assunto por quem já atua na área.

A **Fase 1** cobre a linguagem no nível da expressão e da função: tipos, operadores, controle de

fluxo e as estruturas de dados nativas. A **Fase 2** sobe um nível e trata da organização do código — módulos, pacotes, ambientes, exceções, orientação a objetos e o modelo de dados que faz a sintaxe de Python funcionar como funciona. A **Fase 3** é sobre não ficar sozinho: a biblioteca padrão e as ferramentas de qualidade (testes, tipagem, formatação, documentação). A **Fase 4** aplica tudo isso a quatro domínios concretos — automação e redes, análise de dados, web e bancos de dados. A **Fase 5** vai ao que está embaixo: concorrência, o GIL, memória, desempenho, extensão em C e para onde seguir depois.

As fases 4 e 5 são independentes entre si: quem quer o caminho aplicado pode ir direto da 3 para a 4, e quem quer entender os internos pode pular para a 5, aceitando algumas referências para frente.

Seções fixas de cada capítulo

Todo capítulo traz duas seções obrigatórias ao final:

i Jeito Pythônico — a forma idiomática de fazer o que o capítulo ensinou: o que a comunidade escreve de verdade, qual PEP fundamenta a escolha, qual é o antipadrão equivalente e por que ele machuca.

💡 Laboratório — roteiro prático reproduzível: passos numerados que você executa no seu próprio ambiente, com o resultado esperado de cada um. Termina em algo que funciona.

Sobre o código deste livro

! Todo exemplo roda

Os exemplos foram executados num ambiente virtual limpo antes de entrar no texto, e a saída mostrada é a saída real do interpretador — inclusive nos exemplos de erro, onde o traceback vai completo, porque ler traceback é parte de saber Python.

Onde a saída varia entre execuções (endereço de objeto, ordem de um `set`, data e hora, caminho absoluto), o texto avisa. Onde o exemplo é do que **não** fazer, isso está dito antes do bloco, não depois.

Código que apaga arquivo, executa comando externo ou desserializa dado de origem desconhecida vem sempre precedido do aviso e, quando possível, de um ensaio seguro em diretório descartável.

Licença

Conteúdo sob licença [CC BY-SA 4.0](#). Código e exemplos sob licença MIT. Contribuições, correções e sugestões são bem-vindas via issues no repositório.

Bom proveito — e mantenha um REPL aberto ao lado enquanto lê.

Parte I.

**Volume 1 — Primeiros Passos com
Python**

1. O que é Python: história, filosofia e o Zen da linguagem

Segunda-feira, oito da manhã. O suporte quer saber quais clientes caíram mais vezes durante o fim de semana, porque três deles ligaram reclamando e ninguém sabe se o problema é da rede ou do roteador da casa. O arquivo com os eventos de conexão está ali, com uma linha por evento, e tem alguns milhares delas. Abrir isso numa planilha é possível, mas você vai fazer de novo na semana que vem, e na outra.

Com Python, a resposta cabe em seis linhas:

```
from collections import Counter
from pathlib import Path

linhas = Path("conexoes.log").read_text(encoding="utf-8").splitlines()
quedas = Counter(l.split()[2] for l in linhas if l.endswith("DOWN"))

for cliente, total in quedas.most_common(3):
    print(f"{cliente:<20} {total} quedas")
```

```
sitio-mangueira      4 quedas
fazenda-boia-vista  3 quedas
chacara-do-ze       2 quedas
```

Este capítulo é sobre a linguagem que permite escrever essas seis linhas — de onde ela veio, quem decide como ela cresce e qual é o conjunto de valores que faz um programa Python parecer com esse e não com outra coisa.

1.1. Seis linhas e vinte e quatro

O trecho acima não é curto por esperteza. Ele é curto porque a linguagem já tem palavras para as três coisas que o problema pede: *ler um arquivo de texto* (`Path.read_text`), *contar ocorrências* (`Counter`) e *pegar as maiores* (`most_common`). Escrito como quem transliterou o programa de outra linguagem, o mesmo resultado sai assim:

```
arquivo = open("conexoes.log")
linhas = arquivo.readlines()
arquivo.close()

quedas = {}
i = 0
while i < len(linhas):
    linha = linhas[i]
```

1. O que é Python: história, filosofia e o Zen da linguagem

```
partes = linha.split()
if len(partes) == 4 and partes[3] == "DOWN":
    cliente = partes[2]
    if cliente in quedas:
        quedas[cliente] = quedas[cliente] + 1
    else:
        quedas[cliente] = 1
i = i + 1

pares = []
for cliente in quedas:
    pares.append((quedas[cliente], cliente))
pares.sort()
pares.reverse()

j = 0
while j < 3 and j < len(pares):
    print(pares[j][1] + " " * (21 - len(pares[j][1])) + str(pares[j][0]) + "
↪ quedas")
    j = j + 1
```

São 24 linhas de código contra 6, e a saída é byte a byte a mesma. As duas versões rodam; a segunda apenas gasta o leitor. Ela também esconde três bugs em potencial que a primeira nem tem chance de cometer: o arquivo que fica aberto se algo estourar no meio, o índice `i` que alguém esquece de incrementar num `if`, e o 21 mágico da formatação manual.

A diferença entre as duas não é uma questão de gosto. É a consequência prática de uma decisão de projeto tomada em 1989 e defendida com teimosia desde então: **o código é lido muito mais vezes do que é escrito**, e a linguagem deve otimizar para a leitura.

1.2. De um projeto de fim de ano a uma linguagem de infraestrutura

Em dezembro de 1989, Guido van Rossum trabalhava no CWI — o Centrum voor Wiskunde en Informatica, o instituto nacional holandês de matemática e computação, em Amsterdã. A equipe dele mantinha o Amoeba, um sistema operacional distribuído, e enfrentava um problema chato: escrever utilitários de administração em C era lento demais para o tamanho da tarefa, e o shell não dava conta de nada que exigisse estrutura de dados de verdade.

Van Rossum tinha acabado de sair de outro projeto do mesmo instituto: a linguagem ABC, pensada para ensinar programação a não programadores. O ABC acertou em coisas que ninguém mais fazia — blocos delimitados por indentação, tipos de alto nível embutidos, sintaxe legível em voz alta — e errou em outras, de forma fatal: era monolítica, praticamente impossível de estender, e não conversava com o sistema operacional. Você não conseguia abrir um arquivo do jeito que o sistema queria, nem chamar uma biblioteca em C.

O projeto que ele começou naquele fim de ano era, em essência, o ABC com as arestas corrigidas: a mesma legibilidade, mas extensível, capaz de chamar código C e de falar com o sistema. O nome não veio da cobra. Van Rossum era fã do grupo de humor britânico Monty Python, e “Python” entrou como título provisório de um arquivo. Nunca foi trocado.

A primeira versão pública, a 0.9.0, foi postada em fevereiro de 1991 no grupo `alt.sources` da Usenet — já com classes, herança, tratamento de exceções, funções e os tipos `list` e `dict`. A 1.0 saiu em janeiro de 1994. A 2.0, em outubro de 2000, trouxe as compreensões de lista e um coletor de lixo capaz de resolver ciclos de referência. Em 2001 foi criada a Python Software Foundation, que passou a deter a propriedade intelectual do projeto — uma decisão burocrática e sem glamour que provavelmente é uma das razões de a linguagem ainda existir. A Figura 1.1 situa esses marcos.

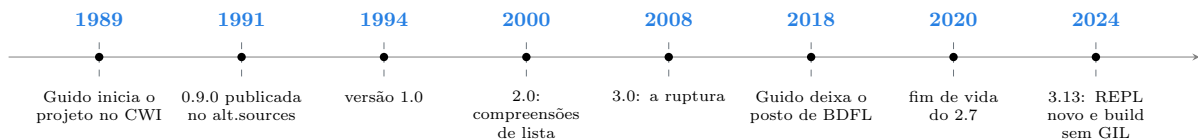


Figura 1.1.: Trinta e cinco anos de Python em oito marcos.

1.3. A ruptura do Python 3 e a lição que ficou

Em dezembro de 2008 saiu o Python 3.0, e ele quebrou compatibilidade de propósito. Foi a decisão mais cara da história do projeto e a mais defensável.

O motivo central era texto. No Python 2, o tipo `str` era uma sequência de bytes que fingia ser texto, e o texto de verdade morava num tipo separado, `unicode`. Enquanto todo mundo escrevia em inglês ASCII, dava para não notar. Assim que entrava um acento — um nome de cliente como “Fazenda Boa Vista” vindo de um sistema em ISO-8859-1, ou um log gerado por um equipamento que fala UTF-8 — o programa passava a misturar bytes e texto sem avisar, e o resultado eram aqueles caracteres embaralhados que todo mundo já viu, ou uma exceção estourando em produção três meses depois, na única linha em que o acento aparecia.

O Python 3 separou os dois de forma inegociável: `str` é texto, `bytes` é byte, e a conversão entre eles exige que você diga qual é a codificação. É por isso que o exemplo da abertura escreve `encoding="utf-8"` em vez de deixar o padrão decidir. Parece rigor excessivo até o dia em que salva o seu relatório.

A travessia foi longa e dolorosa. Por mais de uma década existiu código de produção nas duas versões, bibliotecas que só funcionavam numa delas e distribuições Linux que traziam `python` apontando para o 2.7 por medo de quebrar scripts de sistema. O suporte ao Python 2.7 terminou em 1º de janeiro de 2020, onze anos depois do lançamento do 3.0.

Para quem começa hoje, isso é história — mas história com uma consequência prática: **ao procurar solução na internet, você ainda vai esbarrar em respostas escritas para o Python 2**. O sinal mais rápido é o `print` sem parênteses: `print "olá"` é Python 2 e é erro de sintaxe hoje. Nas distribuições atuais, incluindo o Kali, o comando `python` já aponta para o Python 3, e `python2` simplesmente não vem instalado.

1.4. Quem decide o que entra na linguagem

Mudanças no Python não acontecem por decisão isolada de quem tem acesso ao repositório. Elas passam por uma PEP — *Python Enhancement Proposal*, proposta de melhoria do Python —, um documento numerado que descreve a mudança, a motivação, a especificação técnica, as

1. O que é Python: história, filosofia e o Zen da linguagem

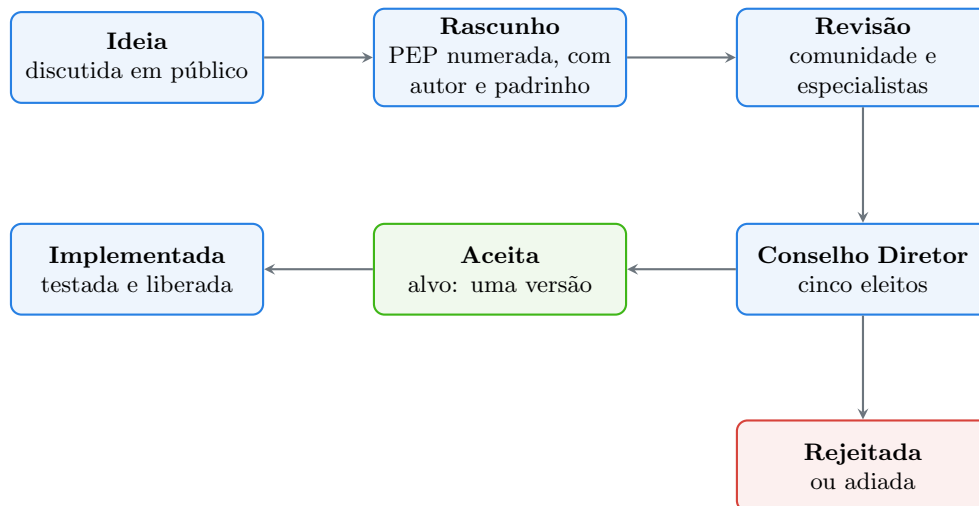


Figura 1.2.: O caminho de uma mudança na linguagem, da ideia à liberação.

alternativas consideradas e o impacto sobre código existente. O processo é definido pela PEP 1, e o fluxo está em Figura 1.2.

Por quase trinta anos, a palavra final era de Guido van Rossum, com o título semi-brincalhão de BDFL — *Benevolent Dictator For Life*, ditador benevolente vitalício. O arranjo acabou em julho de 2018. A PEP 572, que introduziu o operador morsa (`:=`), gerou uma discussão longa e áspera; Van Rossum aprovou a proposta e, dias depois, anunciou que estava se afastando do papel de árbitro. A comunidade respondeu com a PEP 13, que criou o Conselho Diretor: cinco pessoas eleitas anualmente pelos desenvolvedores do núcleo, com mandato coletivo sobre as decisões técnicas.

Vale registrar que o operador morsa, motivo de tanto barulho, hoje é usado sem drama — e que Van Rossum continuou contribuindo com o projeto depois de sair do posto. A instituição sobreviveu à saída do fundador, que é exatamente o que se espera de uma linguagem em que as pessoas vão apostar a infraestrutura delas.

A PEP 602 estabeleceu o ritmo atual de lançamentos: uma versão nova por ano, sempre em outubro. Cada versão recebe cerca de dois anos de correções de bugs e mais três de correções apenas de segurança — cinco anos de vida útil, previsíveis de antemão. Isso permite planejar migração em vez de reagir a ela.

1.5. Python 3.13, a versão de referência deste manual

Este manual usa o **Python 3.13**, lançado em outubro de 2024. É a versão em que todos os exemplos foram executados antes de a saída ser colada aqui. Especificamente:

```
$ python --version
Python 3.13.15
```

O que o 3.13 trouxe de mais visível:

- **Um REPL novo.** O modo interativo ganhou histórico multilinha de verdade, colagem de blocos inteiros sem bagunçar a indentação, cores na saída de erro e comandos como

`help`, `exit` e `clear` funcionando sem parênteses. Quem usava o IPython só por causa disso talvez não precise mais.

- **Mensagens de erro melhores.** A linha do traceback agora aponta com `~` e `^` exatamente qual pedaço da expressão falhou, e não apenas em qual linha.
- **Um build experimental sem GIL** (PEP 703), instalado à parte como `python3.13t`, em que threads de Python realmente rodam em paralelo. É o começo de um trabalho de vários anos; nas versões seguintes ele deixou de ser experimental. O assunto tem um volume inteiro reservado neste manual.
- **Um compilador JIT experimental** (PEP 744), desligado por padrão.
- **A remoção das “pilhas mortas”** (PEP 594): dezenove módulos obsoletos da biblioteca padrão saíram de vez, entre eles `telnetlib`, `cgi` e `crypt`.

Nada disso muda a linguagem que você vai aprender nos próximos capítulos. Muda o conforto de usá-la.

1.6. O Zen do Python

Em 1999, Tim Peters — um dos desenvolvedores mais influentes do núcleo, autor do algoritmo de ordenação que o Python usa até hoje — resumiu numa lista de aforismos os critérios que a comunidade usava para decidir se uma proposta “parecia Python”. A lista virou a PEP 20 e está dentro do interpretador. Digite `import this` em qualquer instalação:

```
import this
```

The Zen of Python, by Tim Peters

```
Beautiful is better than ugly.
Explicit is better than implicit.
Simple is better than complex.
Complex is better than complicated.
Flat is better than nested.
Sparse is better than dense.
Readability counts.
Special cases aren't special enough to break the rules.
Although practicality beats purity.
Errors should never pass silently.
Unless explicitly silenced.
In the face of ambiguity, refuse the temptation to guess.
There should be one-- and preferably only one --obvious way to do it.
Although that way may not be obvious at first unless you're Dutch.
Now is better than never.
Although never is often better than *right* now.
If the implementation is hard to explain, it's a bad idea.
If the implementation is easy to explain, it may be a good idea.
Namespaces are one honking great idea -- let's do more of those!
```

São 19 aforismos. A PEP 20 diz que são 20, e que o vigésimo ficou reservado para Guido preencher — ele nunca preencheu, e a piada permaneceu no documento.

Quatro deles mudam código no dia a dia com mais frequência que os outros.

1. O que é Python: história, filosofia e o Zen da linguagem

“Legibilidade conta.” É a régua para desempatar qualquer discussão de estilo. Se uma linha economiza três caracteres e custa dez segundos de leitura, ela perdeu.

“Explícito é melhor que implícito.” É a razão de o `self` aparecer escrito na assinatura dos métodos, de o `encoding` ser recomendado ao abrir um arquivo de texto e de não existir conversão automática entre número e texto: `"3" + 4` é um erro, não um palpite. Linguagens que adivinham economizam digitação e cobram em depuração.

“Erros nunca devem passar em silêncio — a menos que silenciados explicitamente.” Quando algo dá errado, o Python para e diz onde. Considere um script que lê níveis de sinal óptico e encontra um campo estragado no meio do arquivo:

```
leituras = ["-18.4", "-21.7", "sem leitura", "-19.2"]

for leitura in leituras:
    print(float(leitura))

-18.4
-21.7
Traceback (most recent call last):
  File "C:\Users\Usuario\lab01\sinal.py", line 4, in <module>
    print(float(leitura))
    ~~~~~~^~~~~~
ValueError: could not convert string to float: 'sem leitura'
```

O caminho da primeira linha é o do arquivo na máquina onde o exemplo rodou — no seu, vai aparecer o seu. O que importa é o resto: a linha exata, a expressão exata marcada por `~~~~~^~~~~~`, o tipo do erro e o valor que causou o problema. Nenhum NaN silencioso, nenhum zero inventado no lugar do dado ruim. O programa parou porque continuar seria pior — e é você quem decide o que fazer com o campo estragado, não o interpretador.

“Deve haver uma — e de preferência só uma — maneira óbvia de fazer.” É o aforismo mais citado e o mais mal-entendido. Não quer dizer que exista uma única forma possível; quer dizer que, entre as formas possíveis, a comunidade converge para uma, e essa uma vira a que todo mundo reconhece de imediato. Na prática é isso que permite ler código Python de um projeto que você nunca viu e entender o que ele faz.

O contrapeso está logo acima: **“embora a praticidade vença a pureza”**. O Python aceita romper a regra quando o custo de mantê-la é alto demais. O próprio módulo `this` é a prova:

```
import this

print(this.c, this.i, len(this.d))
```

97 25 52

O texto do Zen está guardado em `this.s` cifrado em ROT13, e o módulo monta a tabela de decifração com dois laços aninhados — deixando as variáveis `c` e `i` vazando no espaço de nomes do módulo. É um deslize que qualquer revisor apontaria, dentro da biblioteca padrão, no arquivo que descreve as boas práticas. Está lá há mais de vinte anos porque não faz mal a ninguém e a piada é boa. Praticidade venceu a pureza.

Já a rigidez sobre a sintaxe, essa é levada a sério, com humor:

```
from __future__ import braces
```

```
File "<string>", line 1
SyntaxError: not a chance
```

Chaves para delimitar blocos, o pedido mais repetido por quem vem de C e de Java, tem uma resposta oficial no interpretador: *nem a pau*.

1.7. Onde Python é forte e onde não é

Ser honesto sobre os limites poupa frustração. O Python é lento em laço numérico puro comparado a C, Rust ou Go — uma ordem de grandeza ou mais. O bloqueio global do interpretador, o GIL, historicamente impediu que threads de Python usassem vários núcleos ao mesmo tempo para trabalho de CPU. Distribuir um programa como um único executável dá trabalho, e a linguagem não é uma escolha natural para aplicativo de celular nem para código embarcado com poucos kilobytes de memória.

Em compensação, ela é imbatível em três situações que cobrem boa parte do trabalho real: **colar sistemas que não foram feitos para conversar** (um log aqui, uma API ali, um banco acolá), **explorar dados** e **automatizar o que hoje é feito na mão**. Nesses casos o gargalo não é o processador — é o disco, a rede ou o tempo de quem escreve o programa. E as partes que precisam de velocidade quase sempre já existem escritas em C por baixo de uma interface Python: é assim que funcionam NumPy, pandas e praticamente todo o ecossistema científico.

Uma distinção que vale desde já: **Python é a linguagem; CPython é o programa que a executa**. O CPython, escrito em C, é a implementação de referência — é ele que você instala e é sobre ele que este manual fala. Existem outras: o PyPy, com compilação em tempo de execução e desempenho muito superior em código de laço; o MicroPython, para microcontroladores; o GraalPy, sobre a máquina virtual da JVM. Todas implementam a mesma linguagem, com graus diferentes de compatibilidade com as extensões em C.

1.8. Jeito Pythônico

“Pythônico” é o adjetivo que a comunidade usa para o código que resolve o problema com o vocabulário da linguagem, e não com o vocabulário de outra linguagem escrito com sintaxe de Python. Ele tem duas fontes escritas: a **PEP 20**, que dá os valores, e a **PEP 8**, o guia de estilo, que dá as convenções concretas — quatro espaços de indentação, **snake_case** para funções e variáveis, **CamelCase** para classes, linhas curtas.

O antipadrão mais comum de quem chega de outra linguagem é percorrer uma sequência pelo índice:

```
# antipadrão: índice manual, herdado de C
i = 0
while i < len(linhas):
    processar(linhas[i])
    i = i + 1
```

1. O que é Python: história, filosofia e o Zen da linguagem

```
# pythônico: iterar sobre o objeto
for linha in linhas:
    processar(linha)
```

A segunda versão não é só mais curta. Ela não tem como errar o limite do laço, funciona igual para lista, arquivo aberto, conjunto ou gerador — e comunica a intenção em vez do mecanismo. Quando o índice é realmente necessário, existe a forma canônica para isso:

```
for numero, linha in enumerate(linhas, start=1):
    if linha.endswith("DOWN"):
        print(f"linha {numero}: {linha}")
```

O mesmo raciocínio vale para “acumular contagem num dicionário” (é `collections.Counter`), “juntar textos com separador” (é `"; ".join(partes)`, não concatenação em laço) e “montar texto com valores dentro” (é a f-string, `f"{cliente}: {total}"`).

Sobre versões, quatro marcos que aparecem em código moderno e que não existiam antes — vale saber a partir de quando cada um pode ser usado:

- **Python 3.8** — o operador morsa, que atribui dentro de uma expressão: `if (n := len(fila)) > 10:`.
- **Python 3.10** — o casamento de padrões, com `match` e `case`, e a barra vertical como união de tipos na anotação, no lugar de `typing.Optional` e `typing.Union`.
- **Python 3.11** — `tomllib` na biblioteca padrão, para ler arquivos TOML sem instalar nada.
- **Python 3.12** — a sintaxe nova de genéricos e de alias de tipo, com `type` e colchetes na assinatura.

Se o seu código precisa rodar num servidor com Python 3.9, `match` está fora — e é para isso que a versão mínima serve. Cada um desses recursos tem seu capítulo mais adiante.

Por fim, uma regra que resume o resto: **escreva para quem vai ler o código daqui a seis meses, que provavelmente é você e não vai lembrar de nada**. Um nome de variável de dez letras que diz o que a variável é vale mais do que um comentário explicando um nome de uma letra.

1.9. Laboratório

Objetivo: montar um ambiente isolado com Python 3.13, confirmar a versão, explorar o Zen por dentro e rodar o script da abertura com dados próprios. Tudo com a biblioteca padrão — nada a instalar.

1. Crie o diretório e o ambiente virtual. No Linux (Kali, Debian, Ubuntu):

```
mkdir -p ~/lab01 && cd ~/lab01
python3 -m venv .venv
source .venv/bin/activate
```

No Windows, no PowerShell, as duas últimas linhas viram:

```
py -3 -m venv .venv
.\.venv\Scripts\Activate.ps1
```

O prompt passa a exibir (.venv) na frente. A partir daqui, python é o do ambiente.

2. Confirme a versão. Esperado: 3.13 alguma coisa. O número após o segundo ponto varia conforme a data.

```
python --version
python -c "import sys; print(sys.version_info)"
```

```
Python 3.13.15
sys.version_info(major=3, minor=13, micro=15, releaselevel='final', serial=0)
```

Se aparecer 3.9 ou 2.7, o ambiente não foi ativado ou a versão instalada é outra. O capítulo sobre instalação, mais adiante no Volume 1, trata dos gerenciadores de versão.

3. Leia o Zen. Ainda no terminal:

```
python -c "import this"
```

A saída é a lista de 19 aforismos mostrada neste capítulo.

4. Olhe o Zen por dentro. Crie o arquivo zen.py:

```
import this

print(this.c, this.i, len(this.d))
print(this.s.splitlines()[13])
print("".join(this.d.get(ch, ch) for ch in this.s.splitlines()[13]))
```

Rode com `python zen.py`. Depois das 19 linhas impressas pelo `import`, sai isto:

```
97 25 52
Va gur snpr bs nzovthvgl, ershfr gur grzcgngvba gb thrff.
In the face of ambiguity, refuse the temptation to guess.
```

Três coisas para notar. As variáveis `c` e `i` sobreviveram aos laços do módulo — é o vazamento comentado acima. O dicionário `d` tem 52 entradas: 26 letras maiúsculas mais 26 minúsculas. E a última linha decifra o aforismo aplicando a mesma tabela: ROT13 é a sua própria inversa.

5. Gere um log de exemplo. Crie `conexoes.log` com este conteúdo — são pares de queda e retorno de quatro clientes ao longo de um dia:

1. O que é Python: história, filosofia e o Zen da linguagem

```
2026-03-11 03:14:02 sitio-mangueira DOWN
2026-03-11 03:14:41 sitio-mangueira UP
2026-03-11 05:02:19 fazenda-boavista DOWN
2026-03-11 05:03:07 fazenda-boavista UP
2026-03-11 06:47:55 sitio-mangueira DOWN
2026-03-11 06:48:30 sitio-mangueira UP
2026-03-11 09:21:10 chacara-do-ze DOWN
2026-03-11 09:22:02 chacara-do-ze UP
2026-03-11 11:05:44 sitio-mangueira DOWN
2026-03-11 11:06:21 sitio-mangueira UP
2026-03-11 12:44:38 fazenda-boavista DOWN
2026-03-11 12:45:26 fazenda-boavista UP
2026-03-11 14:38:09 chacara-do-ze DOWN
2026-03-11 14:39:00 chacara-do-ze UP
2026-03-11 17:12:47 fazenda-boavista DOWN
2026-03-11 17:13:35 fazenda-boavista UP
2026-03-11 18:52:31 sitio-mangueira DOWN
2026-03-11 18:53:12 sitio-mangueira UP
2026-03-11 21:30:05 vila-sao-joao DOWN
2026-03-11 21:30:52 vila-sao-joao UP
```

6. Rode o script da abertura. Salve como `quedas.py` o trecho de seis linhas do começo do capítulo e execute:

```
python quedas.py
```

```
sitio-mangueira      4 quedas
fazenda-boavista     3 quedas
chacara-do-ze        2 quedas
```

7. Provoque um erro de propósito. Crie `sinal.py` com o exemplo dos níveis de sinal e rode. Você deve ver o `ValueError` com o traceback apontando a linha 4 e a expressão `float(leitura)`. Leia a última linha do traceback antes de qualquer outra coisa: é ela que diz o que aconteceu. Ler traceback é a habilidade que mais economiza tempo em Python, e tem um capítulo só para isso ainda no Volume 1.

8. Peça as chaves. Para fechar:

```
python -c "from __future__ import braces"
```

```
File "<string>", line 1
SyntaxError: not a chance
```

O número da linha muda conforme a forma de execução — dentro de um arquivo, ele aponta a linha do arquivo.

9. Saia do ambiente com `deactivate`. O diretório `~/lab01` fica para os próximos capítulos.

1.10. Resumo

- Python nasceu em dezembro de 1989, no CWI, em Amsterdã, como um ABC corrigido: a mesma legibilidade, mas extensível e capaz de falar com o sistema operacional. O nome vem do Monty Python.
- A ruptura do Python 3, em 2008, existiu principalmente para separar texto de bytes de forma inegociável. Custou uma década de transição e terminou em 2020, com o fim de vida do 2.7.
- Mudanças na linguagem passam por PEPs. O poder de decisão saiu do BDFL em 2018 e hoje está com um Conselho Diretor de cinco pessoas eleitas; desde a PEP 602 sai uma versão por ano, em outubro, com cinco anos de vida útil.
- A versão de referência deste manual é a 3.13, com REPL novo, mensagens de erro mais precisas e os primeiros passos do interpretador sem GIL.
- O Zen do Python (PEP 20, `import this`) resume os valores: legibilidade conta, explícito vence implícito, erro não passa em silêncio, e há uma maneira óbvia — embora a praticidade vença a pureza quando o custo da pureza é alto.
- Código pythônico usa o vocabulário da linguagem: iterar sobre o objeto em vez de indexar, `Counter` em vez de contar na mão, f-string em vez de concatenar. Seis linhas legíveis valem mais que vinte e quatro literais.

2. Como o Python executa código: interpretador, bytecode e CPython

Você entrega um script para um colega e ele volta com duas perguntas. A primeira: “apareceu uma pasta `__pycache__` no meio do projeto, posso apagar?”. A segunda é mais interessante — ele digitou o script errado, com um `+` sobrando no fim de uma função que o programa nem chega a chamar, e nada rodou. Nem a primeira linha, que era um `print`. Se o Python é interpretado linha por linha, por que a primeira linha não executou?

Porque ele não é interpretado linha por linha. Entre o arquivo `.py` e o resultado na tela existem quatro etapas, e saber quais são explica a pasta misteriosa, o erro que aparece antes de qualquer saída, o motivo de o segundo `import` de um módulo ser mais rápido que o primeiro e boa parte do vocabulário que você vai encontrar em discussão sobre desempenho de Python.

2.1. O erro que chega antes da primeira linha

Vamos reproduzir a queixa do colega. Salve isto como `sintaxe.py`:

```
print("esta linha nunca vai aparecer")

def nunca_chamada():
    return 1 +
```

A função com o `+` solto nunca é chamada. Ainda assim:

```
File "C:\Users\Usuario\lab02\sintaxe.py", line 5
    return 1 +
            ^
SyntaxError: invalid syntax
```

O `print` não rodou. Agora troque o erro de sintaxe por um erro de nome — uma variável que não existe, também dentro da função que ninguém chama. Salve como `nome.py`:

```
print("esta linha aparece")

def nunca_chamada():
    return taxa_de_juros * 2
```

```
esta linha aparece
```

2. Como o Python executa código: interpretador, bytecode e CPython

Nenhum erro. A variável `taxa_de_juros` não existe em lugar nenhum, e o programa terminou com sucesso.

A diferença entre os dois casos é a diferença entre as duas fases da execução. Antes de rodar qualquer coisa, o Python lê o arquivo **inteiro** e o traduz para uma forma interna. Um + sobrando impede a tradução: não há como traduzir o que não é Python válido, então nada roda. Já `taxa_de_juros` é sintaticamente perfeito — é um nome, e nomes são resolvidos no momento em que a linha executa. Como aquela linha nunca executou, ninguém foi procurar a variável.

Essa é a regra prática que vale guardar: **erro de sintaxe é detectado antes de o programa começar; quase todo o resto só aparece quando a linha é alcançada**. É por isso que um `if` com um trecho errado num galho raro pode ficar meses em produção sem ninguém notar, e por isso existem verificadores estáticos e testes — assunto do Volume 11.

2.2. As quatro etapas

O caminho do arquivo até o resultado está em Figura 2.1.

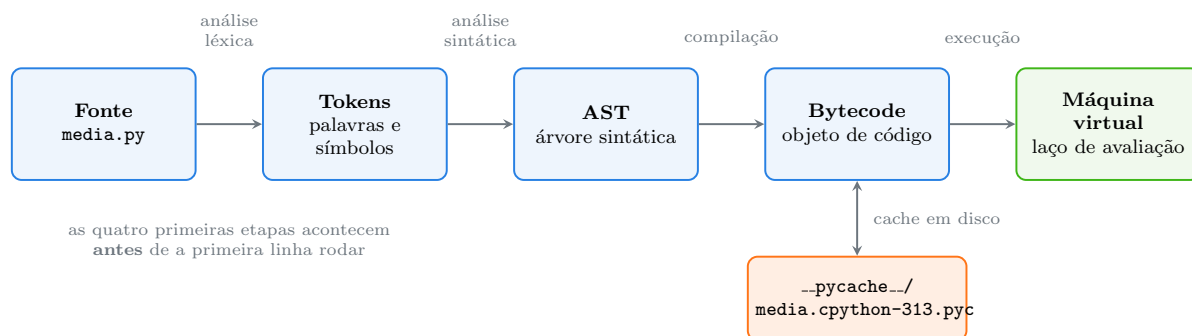


Figura 2.1.: Do arquivo `.py` ao resultado: quatro etapas e um cache.

Análise léxica. O texto do arquivo é quebrado em *tokens* — as menores unidades com significado. A linha `total = preco * 3` vira seis tokens: o nome `total`, o sinal de igual, o nome `preco`, o asterisco, o número 3 e o fim de linha. É nesta etapa que a indentação é convertida em tokens de abertura e fechamento de bloco, o que faz do espaço em branco uma parte da gramática — tema do Capítulo 6.

Análise sintática. Os tokens são organizados numa árvore que representa a estrutura do programa: a *AST*, de *Abstract Syntax Tree*, árvore sintática abstrata. É aqui que o + solto do exemplo anterior derruba tudo: a gramática do Python não tem nenhuma forma válida em que um + seja seguido de fim de linha dentro de um `return`. Você pode ver a árvore com o módulo `ast`:

```
import ast

arvore = ast.parse("total = preco * 3")
print(ast.dump(arvore, indent=2))
```

```
Module(
  body=[
    Assign(
```

```

targets=[
    Name(id='total', ctx=Store())],
value=BinOp(
    left=Name(id='preco', ctx=Load()),
    op=Mult(),
    right=Constant(value=3))))

```

Leia de dentro para fora: uma operação binária (**BinOp**) de multiplicação (**Mult**) entre o nome **preco**, lido (**Load**), e a constante 3; o resultado é atribuído (**Assign**) ao nome **total**, agora escrito (**Store**). A distinção entre **Load** e **Store** no mesmo tipo de nó é o que diferencia ler de uma variável e escrever nela.

Compilação. A árvore é traduzida para *bytecode*: uma sequência de instruções simples para uma máquina imaginária. O bytecode não é código de máquina — nenhum processador real sabe executá-lo. Ele fica guardado num **objeto de código**, que é um valor de primeira classe em Python, como um número ou uma lista:

```

codigo = compile("preco = 19.9\ntotal = preco * 3", "<exemplo>", "exec")
print(type(codigo))
print(codigo.co_consts)
print(codigo.co_names)

```

```

<class 'code'>
(19.9, 3, None)
('preco', 'total')

```

O objeto de código carrega as constantes que aparecem no trecho (**co_consts**) e os nomes que ele usa (**co_names**), separados. É o formato que a máquina virtual consome.

Execução. A máquina virtual — na prática, um laço em C que lê uma instrução, faz o que ela manda e passa para a seguinte — avalia o bytecode. É esse laço que as pessoas chamam de “o interpretador”, e é dele que vem a fama de lentidão do Python: cada operação que em C seria uma instrução de processador aqui custa uma volta no laço, com desvio, verificação de tipo e contagem de referências.

Então o Python é compilado ou interpretado? As duas coisas, e a pergunta é mal colocada. Ele é **compilado para bytecode** e **o bytecode é interpretado**. Java faz o mesmo. A diferença prática em relação a C é que a compilação acontece automaticamente, na hora de rodar, e não numa etapa separada que produz um executável.

2.3. O bytecode por dentro

O módulo **dis** — de *disassemble*, desmontar — mostra o bytecode em forma legível. Crie **media.py**:

```

def media(valores):
    total = 0
    for v in valores:
        total += v
    return total / len(valores)

```

2. Como o Python executa código: interpretador, bytecode e CPython

E `ver.py`:

```
import dis
from media import media

dis.dis(media)
```

1	RESUME	0
2	LOAD_CONST	1 (0)
	STORE_FAST	1 (total)
3	LOAD_FAST	0 (valores)
	GET_ITER	
L1:	FOR_ITER	7 (to L2)
	STORE_FAST	2 (v)
4	LOAD_FAST_LOAD_FAST	18 (total, v)
	BINARY_OP	13 (+=)
	STORE_FAST	1 (total)
	JUMP_BACKWARD	9 (to L1)
3	L2: END_FOR	
	POP_TOP	
5	LOAD_FAST	1 (total)
	LOAD_GLOBAL	1 (len + NULL)
	LOAD_FAST	0 (valores)
	CALL	1
	BINARY_OP	11 (/)
	RETURN_VALUE	

A primeira coluna é a linha do arquivo original — é assim que o traceback consegue dizer em que linha o erro aconteceu, assunto do Capítulo 7. Depois vem o nome da instrução, o argumento numérico e, entre parênteses, o que aquele número significa.

Duas coisas para notar. A primeira é `LOAD_FAST` contra `LOAD_GLOBAL`: variáveis locais de uma função moram num vetor de tamanho fixo, acessado por posição, enquanto `len` é procurada num dicionário de nomes globais e depois nos embutidos. É a razão técnica de a busca de local ser mais rápida que a de global, e de o truque antigo de copiar uma função global para uma variável local em laço apertado às vezes ainda render alguma coisa.

A segunda é `LOAD_FAST_LOAD_FAST`, que carrega duas variáveis locais numa única instrução. Não existe nada disso no código-fonte: é o compilador juntando duas operações frequentes numa só para gastar menos voltas no laço de avaliação. Instruções combinadas assim vêm crescendo desde o Python 3.11, e é boa parte do ganho de velocidade dessas versões.

2.3.1. A máquina é uma pilha

O detalhe que torna o bytecode legível é que a máquina virtual do CPython é uma **máquina de pilha**: não existem registradores nomeados, apenas uma pilha de valores. Instrução que começa

com `LOAD` empilha; operação consome do topo e devolve o resultado ao topo; `STORE` retira do topo e guarda num nome. Figura 2.2 acompanha a avaliação de `total = preco * 3`.

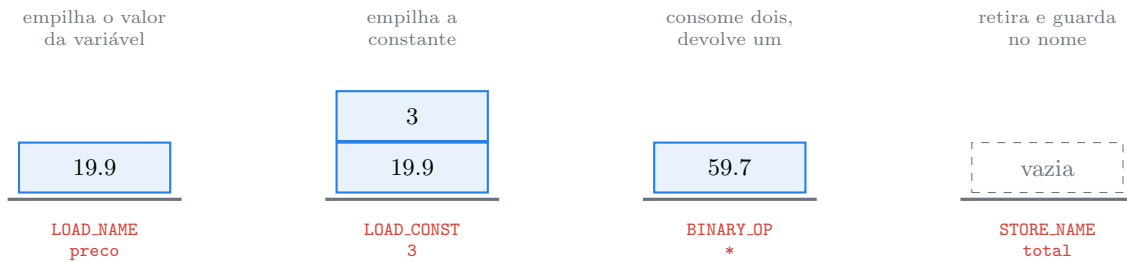


Figura 2.2.: A pilha de avaliação durante `total = preco * 3`, com `preco` valendo 19.9. A pilha cresce para cima; a linha grossa é o fundo.

Quatro instruções, quatro estados da pilha, e no fim ela volta a ficar vazia. Toda expressão Python, por complicada que seja, é decomposta nesse vaivém.

2.3.2. O compilador faz pouca coisa — mas faz

O compilador do CPython não é um otimizador agressivo, e não pretende ser. Ainda assim, algumas simplificações óbvias ele resolve na compilação, de graça:

```
import dis
dis.dis("segundos = 2 * 60 * 60")
```

0	RESUME	0
1	LOAD_CONST	0 (7200)
	STORE_NAME	0 (segundos)
	RETURN_CONST	1 (None)

Não há multiplicação nenhuma no bytecode: o 7200 já está calculado. Isso se chama *constant folding*, dobramento de constantes, e a consequência prática é boa notícia para a legibilidade — escrever `2 * 60 * 60` em vez de 7200 documenta a intenção e **não custa nada em tempo de execução**. Vale para expressões entre literais. Não vale se qualquer operando for uma variável, porque aí o valor só é conhecido na hora.

O que o compilador **não** faz é reorganizar o seu algoritmo. Compare montar um texto concatenando em laço com montar usando `join`:

```
python -m timeit -s "partes=['abc']*2000" "s="
for p in partes: s+=p
python -m timeit -s "partes=['abc']*2000" "s=''.join(partes)"
```

```
2000 loops, best of 5: 259 usec per loop
50000 loops, best of 5: 6.77 usec per loop
```

2. Como o Python executa código: interpretador, bytecode e CPython

Quase quarenta vezes de diferença, e os números na sua máquina vão ser outros — o que se mantém é a ordem de grandeza. Nenhuma otimização de bytecode salva o primeiro caso, porque o problema não está nas instruções: está em criar dois mil textos intermediários em vez de um. Escolher a estrutura e o algoritmo certos vale mais do que qualquer coisa que se faça no nível da instrução, e é por isso que este manual gasta um volume inteiro em estruturas de dados antes de falar de desempenho.

2.4. `__pycache__`, o cache que aparece sozinho

Compilar custa tempo. Fazer isso a cada execução de um módulo que não mudou seria desperdício, então o CPython guarda o resultado. Depois de rodar `ver.py`, o diretório tem uma pasta nova:

```
ls __pycache__
```

```
media.cpython-313.pyc
```

Observe **o que não está ali**: não há `ver.cpython-313.pyc`. O arquivo executado diretamente não é cacheado; só os módulos **importados** são. Faz sentido — o script de entrada roda uma vez, o módulo importado tende a ser reaproveitado por vários programas.

O nome do arquivo traz a implementação e a versão (`cpython-313`) justamente para que instalações diferentes de Python convivam no mesmo diretório sem se atropelar. O formato do bytecode muda entre versões menores, e um `.pyc` do 3.12 é ilegível para o 3.13.

O cabeçalho do `.pyc` tem 16 bytes e explica como a invalidação funciona:

```
from pathlib import Path
import struct

dados = Path("__pycache__/media.cpython-313.pyc").read_bytes()
magic, flags, mtime, tam = struct.unpack("<4sIII", dados[:16])
print("magic", magic)
print("flags", flags)
print("mtime no pyc ", mtime)
print("mtime do fonte", int(Path("media.py").stat().st_mtime))
print("tamanho fonte no pyc", tam, "| real", Path("media.py").stat().st_size)
```

```
magic b'\xf3\r\r\n'
flags 0
mtime no pyc    1786393562
mtime do fonte 1786393562
tamanho fonte no pyc 107 | real 107
```

São quatro campos. O *número mágico* identifica a versão do formato de bytecode — se não bater com o do interpretador, o cache é descartado. Os **flags** dizem qual estratégia de invalidação está em uso. E os dois últimos campos são a data de modificação e o tamanho do arquivo-fonte no momento em que o cache foi escrito: a cada **import**, o Python compara esses dois valores com os do `.py` em disco e recompila se algum diferir.

Duas consequências práticas. A primeira é que **você pode apagar `__pycache__` sem medo** — ela é recriada. Responda isso ao colega, e acrescente `__pycache__/` ao `.gitignore`, porque cache não entra no controle de versão. A segunda é mais sutil: a invalidação depende do relógio. Se um processo de build restaura arquivos com data antiga, ou se um contêiner tem relógio deslocado, é possível ficar com bytecode velho rodando fonte novo. Para esses casos existe o modo baseado em hash do conteúdo (PEP 552), ativado por um bit nos `flags`.

Duas variáveis de ambiente e duas opções de linha de comando aparecem em torno disso. `python -B`, ou `PYTHONDONTWRITEBYTECODE=1`, desliga a escrita do cache — comum em imagem de contêiner, onde o `.pyc` só engorda a camada. E `python -O` produz um arquivo com sufixo diferente:

```
python -O -c "import media"
ls __pycache__
```

```
media.cpython-313.opt-1.pyc
media.cpython-313.pyc
```

O que o `-O` faz merece atenção porque o nome engana. Ele não otimiza nada de verdade: apenas remove as instruções `assert` e trecho protegido por `if __debug__`. Compare:

```
python -c "assert False, 'estourou'"
python -O -c "assert False, 'estourou'; print('passou reto')"
```

```
Traceback (most recent call last):
  File "<string>", line 1, in <module>
    assert False, 'estourou'
    ~~~~~
AssertionError: estourou
passou reto
```

Com `-O`, a asserção desaparece. Essa é a razão de nunca usar `assert` para validar entrada de usuário ou verificar permissão: alguém rodando com `-O` some com a sua checagem. `assert` é para condição que você acredita ser sempre verdadeira, como rede de segurança durante o desenvolvimento e nos testes.

2.5. CPython e as outras implementações

Vale repetir a distinção do Capítulo 1, agora com conteúdo técnico: **Python é a linguagem, CPython é o programa que a executa**. O interpretador sabe se apresentar:

```
python -c "import sys; print(sys.implementation.name); print(sys.version)"
```

```
cpython
3.13.15 (main, Aug 7 2026, 02:25:50) [MSC v.1944 64 bit (AMD64)]
```

Tudo que descrevemos até aqui — bytecode de pilha, `__pycache__`, contagem de referências, GIL — é decisão de projeto do CPython, não exigência da linguagem. Outras implementações fazem diferente:

2. Como o Python executa código: interpretador, bytecode e CPython

- **PyPy** compila em tempo de execução, com JIT: ele observa quais trechos rodam muito e gera código de máquina de verdade para eles. Em laço numérico puro chega a ser uma ordem de grandeza mais rápido que o CPython. Em troca, a interface com extensões escritas em C é mais frágil e o consumo de memória é maior.
- **MicroPython** e **CircuitPython** rodam em microcontrolador, com biblioteca padrão reduzida ao que cabe em poucas centenas de kilobytes.
- **GraalPy** roda sobre a máquina virtual da JVM, o que interessa a quem precisa conviver com código Java.

O CPython também está mudando por dentro. O 3.11 trouxe o *interpretador especializado* (PEP 659): a máquina virtual observa os tipos que passam por cada instrução e a substitui por uma versão especializada — um `BINARY_OP` genérico vira uma soma de inteiros direta quando os dois operandos vêm sendo inteiros. É otimização em tempo de execução dentro do interpretador, sem JIT. O 3.13 acrescentou um compilador JIT experimental (PEP 744), desligado por padrão, e o build sem GIL (PEP 703). Nada disso muda o que você escreve; muda quanto tempo leva para rodar. O Volume 15 trata do assunto com o cuidado que ele merece.

Um efeito colateral útil de saber que o `import` custa: quando um programa de linha de comando demora a responder, muitas vezes o culpado é o que ele importa, não o que ele faz. A opção `-X importtime` cobra o tempo de cada `import`, em microssegundos:

```
python -X importtime -c "import json"
```

import time:	1646	4792	re._compiler
import time:	758	758	copyreg
import time:	1864	10631	re
import time:	305	305	_json
import time:	7730	8034	json.scanner
import time:	11253	29918	json.decoder
import time:	7461	7461	json.encoder
import time:	10180	47558	json

A primeira coluna é o tempo do módulo em si; a segunda, o acumulado com tudo que ele arrasta; a indentação mostra quem importou quem. Aqui, importar `json` custou 47 milissegundos, e quase dois terços disso foram gastos em `json.decoder` e no `re` que ele carrega. Numa ferramenta de linha de comando que precisa responder rápido, esse é o tipo de conta que compensa fazer.

2.6. Jeito Pythônico

O erro pythônico mais comum em torno deste assunto é usar o `dis` como árbitro de desempenho. Contar instruções não mede tempo: `LOAD_FAST` e `CALL` aparecem iguais na listagem e custam ordens de grandeza diferentes, e o interpretador especializado troca instruções por outras durante a execução, então a listagem estática nem descreve o que rodou. A forma correta de comparar é medir:

```
# antipadrão: decidir por contagem de instrução no dis
# "a versão A tem 12 instruções e a B tem 14, então A é mais rápida"

# pythônico: medir, com a mesma entrada, no mesmo ambiente
python -m timeit -s "setup" "trecho"
```

O `dis` serve para **entender semântica**, não velocidade: descobrir por que um `for` sobre um dicionário percorre chaves, o que exatamente um decorador acrescentou, por que uma comparação com `is` deu um resultado inesperado. Para velocidade existem o `timeit` e os *profilers*, assunto reservado ao Volume 15.

Três regras que caem direto no dia a dia:

`__pycache__`/ no `.gitignore`, sempre. E jamais distribua `.pyc` no lugar do `.py` como proteção de código: o bytecode é trivial de desmontar com o próprio `dis` da biblioteca padrão, e você amarra o pacote a uma versão exata do interpretador.

`assert` não é validação. Como `-O` remove as asserções, entrada de usuário e verificação de permissão pedem `if` com `raise` explícito:

```
# antipadrão: some com -O
assert idade >= 18, "menor de idade"

# pythônico: sobrevive a qualquer modo de execução
if idade < 18:
    raise ValueError("menor de idade")
```

Prefira legibilidade a pré-cálculo. Como o dobramento de constantes resolve expressão entre literais na compilação, escreva `60 * 60 * 24` e não `86400`; `1024 ** 2` e não `1048576`. Custa zero e explica o número.

Sobre versões, o que muda por aqui:

- **Python 3.11** — interpretador especializante (PEP 659) e `tracebacks` com marcação de coluna. O formato do bytecode mudou bastante; qualquer código que dependa de instruções específicas precisa ser revisto a cada versão.
- **Python 3.12** — `dis` passou a mostrar as instruções combinadas e os saltos com rótulos (`L1`, `L2`), como na saída deste capítulo.
- **Python 3.13** — JIT experimental (PEP 744) e build sem GIL (PEP 703), ambos opcionais.

A moral: **bytecode é detalhe de implementação**. É legítimo e útil olhar para ele para entender o que a linguagem está fazendo. Não é lugar para construir código que dependa de instrução específica, porque isso quebra na próxima versão sem aviso e sem culpa de ninguém.

2.7. Laboratório

Objetivo: observar cada etapa da execução no seu próprio ambiente — ver o erro de compilação chegar antes da execução, desmontar bytecode, decodificar o cabeçalho de um `.pyc` na mão e provocar a invalidação do cache. Só biblioteca padrão.

1. **Prepare o diretório e o ambiente.** No Linux:

```
mkdir -p ~/lab02 && cd ~/lab02
python3 -m venv .venv
source .venv/bin/activate
python --version
```

Python 3.13.15

No Windows, `py -3 -m venv .venv e .\.venv\Scripts\Activate.ps1`.

2. Veja a compilação acontecer antes da execução. Crie `sintaxe.py` e `nome.py` com o conteúdo mostrado no começo do capítulo e rode os dois. O primeiro não imprime nada e falha com `SyntaxError` apontando a linha 5; o segundo imprime a mensagem e termina normalmente, apesar de conter uma variável inexistente. Se você inverter — chamando `nunca_chamada()` no fim do `nome.py` — aí sim aparece o `NameError`, e apenas então.

3. Desmonte uma função. Crie `media.py` e `ver.py` como no capítulo e rode `python ver.py`. Compare a saída com a listagem mostrada aqui. Localize na sua saída: onde o laço volta (`JUMP_BACKWARD`), onde `len` é buscada (`LOAD_GLOBAL`) e onde as locais são lidas (`LOAD_FAST`).

4. Escreva uma variante e compare. Troque o corpo de `media.py` pela versão com `sum`:

```
def media(valores):  
    return sum(valores) / len(valores)
```

Rode `python ver.py` de novo:

1	RESUME	0
2	LOAD_GLOBAL	1 (sum + NULL)
	LOAD_FAST	0 (valores)
	CALL	1
	LOAD_GLOBAL	3 (len + NULL)
	LOAD_FAST	0 (valores)
	CALL	1
	BINARY_OP	11 (/)
	RETURN_VALUE	

Nove instruções contra as dezenove da versão com laço, e o laço em si desapareceu — ele agora acontece dentro de `sum`, escrita em C. Este é o padrão que se repete em Python: empurrar a repetição para dentro de uma função da biblioteca padrão.

5. Encontre o cache. Rode `ls __pycache__` (ou `dir __pycache__` no `cmd`). Deve haver `media.cpython-313.pyc` e **não** deve haver nada referente a `ver.py`. Se a sua versão for outra, o número no nome muda.

6. Decodifique o cabeçalho. Crie `cabecalho.py` com o trecho do `struct.unpack` mostrado no capítulo e rode. Os valores de `mtime` na sua máquina serão outros, mas os dois têm de ser **iguais entre si**, e o tamanho no cabeçalho tem de bater com o tamanho real do `media.py`.

7. Provoque a invalidação. Acrescente uma linha em branco no fim de `media.py`, salve e rode `python cabecalho.py` de novo. Os dois `mtime` agora divergem — o do cache é o antigo. Rode `python ver.py`, que importa o módulo e força a recompilação, e depois `python cabecalho.py` mais uma vez: voltaram a bater, e o tamanho subiu um byte (ou dois, no Windows, por causa do fim de linha).

8. Desligue o cache. Apague a pasta e rode com `-B`:

```
rm -rf __pycache__
python -B ver.py
ls __pycache__
```

A listagem falha porque a pasta não foi criada. Rode sem o `-B` e ela volta.

9. Meça em vez de contar instruções. Reproduza a comparação de concatenação:

```
python -m timeit -s "partes=['abc']*2000" "s=' '
for p in partes: s+=p"
python -m timeit -s "partes=['abc']*2000" "s=' '.join(partes)"
```

Os números absolutos vão diferir dos deste capítulo; a diferença entre eles, não muito.

10. Veja o custo dos imports. Rode `python -X importtime -c "import json"` e depois com `import decimal` e `import asyncio`. O `asyncio` é o mais caro dos três com folga — é bom saber disso antes de importá-lo num script que precisa responder em milissegundos.

11. Feche com deactivate. O diretório `~/lab02` fica para consulta.

2.8. Resumo

- Python não é interpretado linha por linha: o arquivo inteiro é lido, analisado e compilado para bytecode **antes** de qualquer linha rodar. Por isso erro de sintaxe aparece antes do primeiro `print`, e erro de nome só aparece quando a linha é alcançada.
- São quatro etapas: análise léxica (tokens), análise sintática (AST), compilação (bytecode em um objeto de código) e execução na máquina virtual, que é o laço de avaliação escrito em C.
- A máquina virtual do CPython é uma máquina de pilha: `LOAD` empilha, operação consome do topo, `STORE` retira e guarda num nome. O módulo `dis` mostra tudo isso, com a linha do fonte na primeira coluna.
- `__pycache__` guarda o bytecode dos módulos **importados**, com a versão no nome do arquivo. A invalidação compara data e tamanho do fonte gravados no cabeçalho de 16 bytes. Pode apagar, pode ignorar no Git, não serve para esconder código.
- `-O` não otimiza: remove `assert` e `if __debug__`. Logo, `assert` nunca é validação de entrada — para isso, `if` com `raise`.
- Bytecode é detalhe de implementação do CPython, não da linguagem, e muda a cada versão. Use `dis` para entender semântica e `timeit` para medir velocidade; nunca conte instruções para prever desempenho.

3. Instalando o Python: versões, gerenciadores e o Python do sistema

Você precisa de uma biblioteca para consumir uma API. Abre o terminal do Kali, digita `pip install requests` e leva um tapa na cara:

```
error: externally-managed-environment

× This environment is externally managed
> To install Python packages system-wide, try apt install
  python3-xyz, where xyz is the package you are trying to
  install.

  If you wish to install a non-Kali-packaged Python package,
  create a virtual environment using python3 -m venv path/to/venv.
  Then use path/to/venv/bin/python and path/to/venv/bin/pip. Make
  sure you have python3-full installed.

  If you wish to install a non-Kali-packaged Python application,
  it may be easiest to use pipx install xyz, which will manage a
  virtual environment for you. Make sure you have pipx installed.

  For more information, refer to the following:
  * https://www.kali.org/docs/general-use/python3-external-packages/
  * /usr/share/doc/python3.13/README.venv

note: If you believe this is a mistake, please contact your Python installation
↪ or OS distribution provider. You can override this, at the risk of breaking
↪ your Python installation or OS, by passing --break-system-packages.
hint: See PEP 668 for the detailed specification.
```

A tentação é óbvia: a última linha oferece uma saída. `--break-system-packages` resolve na hora, o `requests` instala e você volta ao trabalho. Também é a forma mais rápida conhecida de estragar uma instalação de Linux, e a mensagem se dá o trabalho de dizer isso no nome da opção.

Este capítulo explica por que essa barreira existe, o que fazer em vez de derrubá-la, e como ter a versão do Python que você quer sem entrar em conflito com a que o sistema operacional precisa.

3.1. O Python do sistema não é seu

Numa distribuição Linux moderna, o Python não é um programa que alguém instalou para programar. É uma **peça de infraestrutura**. Ferramentas de administração, o gerenciador de

3. Instalando o Python: versões, gerenciadores e o Python do sistema

pacotes, utilitários de rede e, no caso do Kali, uma parte considerável do arsenal de segurança são escritos em Python e dependem de que o interpretador e as bibliotecas do sistema estejam exatamente na versão que o pacote declarou.

Dá para ver quem está na fila:

```
apt-cache rdepends --installed python3 | head -8
```

```
python3
Reverse Depends:
  python3-dev
  weeveily
  theharvester
  texlive-pictures
  texlive-latex-extra
  |sublist3r
```

`weeveily`, `theHarvester` e `sublist3r` são ferramentas do Kali. Elas não têm ambiente próprio: usam o Python do sistema e as bibliotecas que o `apt` instalou. E são muitas:

```
dpkg -l 'python3-*' | grep -c '^ii'
```

529

Quinhentos e vinte e nove pacotes Python gerenciados pelo `apt`. Quando você roda `pip install requests` com privilégio no interpretador do sistema, o `pip` sobrescreve arquivos que o `apt` considera seus, sem avisar o `apt`. O banco de dados de pacotes passa a mentir: ele diz que a versão instalada é a que veio da distribuição, e o disco tem outra. A partir daí, uma atualização de sistema pode reverter a sua biblioteca, ou a sua biblioteca pode quebrar uma ferramenta que esperava a versão antiga — e o sintoma aparece semanas depois, num programa que você não tocou.

É contra isso que a **PEP 668** existe. Ela padronizou um arquivo marcador que a distribuição coloca ao lado da biblioteca padrão:

```
ls -l /usr/lib/python3.13/EXTERNALLY-MANAGED
```

```
-rw-r--r-- 1 root root 734 Jun  3 04:36 /usr/lib/python3.13/EXTERNALLY-MANAGED
```

O arquivo é um `.ini` cujo campo `Error` é justamente o texto que o `pip` imprimiu:

```
head -4 /usr/lib/python3.13/EXTERNALLY-MANAGED
```

```
[externally-managed]
Error=To install Python packages system-wide, try apt install
python3-xyz, where xyz is the package you are trying to
install.
```


Nenhuma mágica: o `pip` procura esse arquivo e, se o encontra, recusa instalar naquele interpretador. É um acordo entre o empacotador da distribuição e a ferramenta do Python, e a mensagem de erro personalizada é escrita por quem monta a distro. A sua existência não é um bug nem excesso de zelo — é a distribuição dizendo “este interpretador tem dono, e não é você”.

3.2. As três camadas

A saída não é lutar com a barreira, é entender que existem três lugares diferentes onde um Python pode morar, com donos diferentes. Figura 3.1 organiza isso.

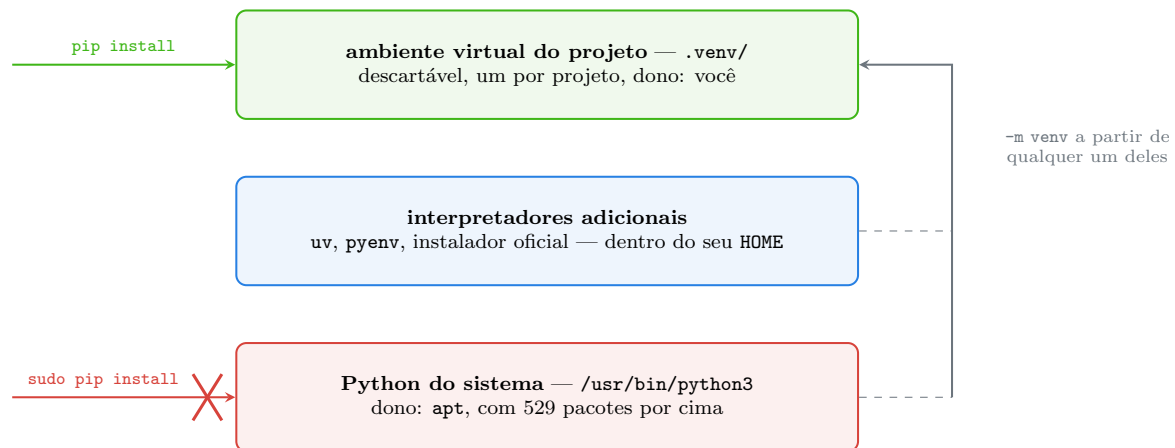


Figura 3.1.: Três camadas, três donos. O `pip` só entra sem pedir licença na de cima.

A camada de baixo é do `apt`. Você lê, executa e não escreve. A do meio é opcional e existe quando você precisa de uma versão que a distribuição não oferece — mora no seu diretório pessoal e não conflita com nada. A de cima é o ambiente virtual, criado a partir de qualquer interpretador das duas camadas de baixo, e é onde o `pip` trabalha à vontade.

3.3. O ambiente virtual, com precisão

Um ambiente virtual é bem menos mágico do que o nome sugere. Crie um e olhe por dentro:

```
mkdir -p /tmp/lab03 && cd /tmp/lab03
python3 -m venv .venv
cat .venv/pyvenv.cfg
```

```
home = /usr/bin
include-system-site-packages = false
version = 3.13.12
executable = /usr/bin/python3.13
command = /usr/bin/python3 -m venv /tmp/lab03/.venv
```

É isso: um arquivo de texto de cinco linhas, um diretório `bin/` com alguns links e um `lib/python3.13/site-packages/` vazio. Não há cópia do interpretador — `home` e `executable` apontam para o Python de verdade, lá na camada de baixo. O que muda é onde ele procura pacotes:

3. Instalando o Python: versões, gerenciadores e o Python do sistema

```
.venv/bin/python -c "import sys; print('prefix      ', sys.prefix);  
↪ print('base_prefix', sys.base_prefix)"
```

```
prefix      /tmp/lab03/.venv  
base_prefix /usr
```

`sys.base_prefix` é o interpretador real; `sys.prefix` é o ambiente. Quando os dois diferem, você está num venv — e é exatamente assim que ferramentas detectam isso. O efeito prático aparece no caminho de busca:

```
.venv/bin/python -c "import sys; [print(p) for p in sys.path if p]"
```

```
/usr/lib/python313.zip  
/usr/lib/python3.13  
/usr/lib/python3.13/lib-dynload  
/tmp/lab03/.venv/lib/python3.13/site-packages
```

A biblioteca padrão continua vindo de `/usr/lib/python3.13` — ela é compartilhada, e não faria sentido duplicá-la. O que sumiu da lista é o `/usr/lib/python3/dist-packages`, onde moram os 529 pacotes do `apt`. No lugar dele entrou o `site-packages` do projeto. Por isso o `pip` pode trabalhar aqui sem incomodar ninguém:

```
.venv/bin/python -m pip install requests cowsay  
.venv/bin/python -m pip list
```

Package	Version
certifi	2026.7.22
charset-normalizer	3.4.9
cowsay	6.1
humanize	4.16.0
idna	3.18
pip	26.1.1
requests	2.34.2
urllib3	2.7.0

E o isolamento é verificável nos dois sentidos. Um pacote que só existe no ambiente não vaza para o sistema:

```
.venv/bin/python -c "import cowsay; print(cowsay.__file__)"  
python3 -c "import cowsay"
```

```
/tmp/lab03/.venv/lib/python3.13/site-packages/cowsay/__init__.py  
import cowsay  
ModuleNotFoundError: No module named 'cowsay'
```

E um pacote que existe nos dois lados fica em duas versões independentes, cada interpretador vendo a sua:

```
python3 -c "import requests; print(requests.__version__, requests.__file__)"
.venv/bin/python -c "import requests; print(requests.__version__,
↪ requests.__file__)"
```

```
2.32.5 /usr/lib/python3/dist-packages/requests/__init__.py
2.34.2 /tmp/lab03/.venv/lib/python3.13/site-packages/requests/__init__.py
```

O Kali traz `requests` 2.32.5 porque é a versão que as ferramentas dele foram testadas contra. O seu projeto usa a 2.34.2. As duas convivem, ninguém quebra, e você não precisou pedir licença. **Este é o argumento inteiro a favor do ambiente virtual**, e ele vale muito mais do que a economia de espaço em disco que às vezes se cita.

3.3.1. O que `activate` faz de verdade

Chamar `.venv/bin/python` pelo caminho completo funciona sempre e é o que scripts e ferramentas fazem. Para trabalho interativo existe a ativação, que costuma ser descrita como um ritual obscuro e não é nada disso:

```
which python3
source .venv/bin/activate
which python
python --version
```

```
/usr/bin/python3
/tmp/lab03/.venv/bin/python
Python 3.13.12
```

A única coisa relevante que aconteceu está em Figura 3.2: o diretório `bin/` do ambiente foi colocado **na frente** do `PATH`.

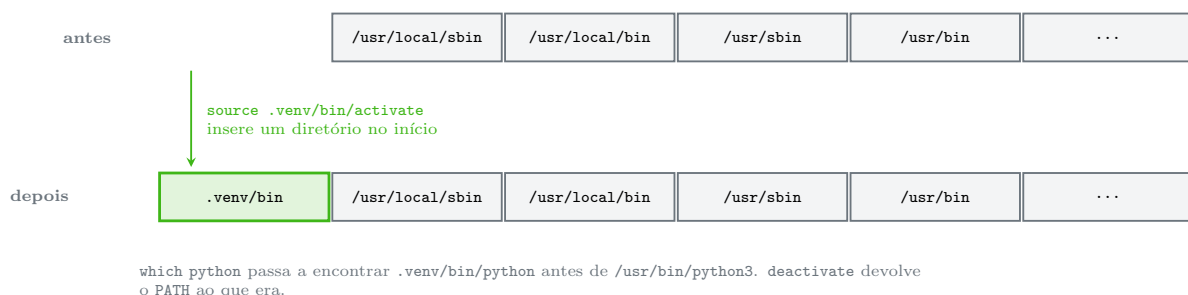


Figura 3.2.: Ativar um ambiente virtual é inserir um diretório no começo do `PATH`.

Nada mais acontece de essencial. O script também ajusta o prompt para mostrar `(.venv)` e define uma função `deactivate` que desfaz a mudança. Não há processo em segundo plano, nada é gravado fora do diretório, e fechar o terminal já desativa. Uma consequência prática útil: como é só `PATH`, ativação **não** é herdada por um processo que você lançou antes, e `sudo` não a preserva — `sudo pip install` dentro de um `venv` ativado instala no Python do sistema, que é o oposto do que você queria.

3.4. Quando você precisa de outra versão

Até aqui, o venv resolveu o problema dos **pacotes**. Ele não resolve o problema da **versão do interpretador**: o `pyvenv.cfg` acima diz `version = 3.13.12` porque foi criado a partir do Python do sistema, e não há como um venv oferecer um 3.11 se a máquina só tem 3.13.

Três situações pedem outra versão. Um projeto de trabalho preso a um servidor com 3.9. Uma biblioteca que ainda não suporta a versão mais nova. E o inverso: você quer testar um recurso de 3.14 numa máquina cuja distribuição parou no 3.11. Para todos os casos, a resposta é a camada do meio da Figura 3.1 — interpretadores instalados no seu diretório pessoal, sem `sudo` e sem tocar no sistema.

uv é hoje o caminho mais direto. Ele é um gerenciador de pacotes e de versões escrito em Rust, resolve dependências rápido e baixa interpretadores prontos:

```
uv python list
```

```
cpython-3.15.0rc1-windows-x86_64-none          <download available>
cpython-3.15.0rc1+freethreaded-windows-x86_64-none  <download available>
cpython-3.14.7-windows-x86_64-none             <download available>
cpython-3.14.7+freethreaded-windows-x86_64-none  <download available>
cpython-3.14.4-windows-x86_64-none             C:\Python314\python.exe
cpython-3.13.15-windows-x86_64-none
  ↳ C:\Users\Usuario\AppData\Roaming\uv\python\cpython-3.13.15-windows-x86_64-none\python.exe
cpython-3.13.15+freethreaded-windows-x86_64-none  <download available>
cpython-3.12.13-windows-x86_64-none             <download available>
cpython-3.11.15-windows-x86_64-none             <download available>
cpython-3.10.20-windows-x86_64-none             <download available>
cpython-3.9.25-windows-x86_64-none              <download available>
pypy-3.11.15-windows-x86_64-none                <download available>
graalpy-3.12.0-windows-x86_64-none              <download available>
```

A listagem foi tirada de uma máquina Windows, e é por isso que aparece `windows-x86_64`; no Linux os nomes trazem `linux-x86_64-gnu`. O que interessa é a estrutura: cada linha é uma combinação de implementação, versão e plataforma, e a coluna da direita diz se já está em disco ou se é download. Repare que estão ali o **freethreaded** — o build sem GIL de que falamos no Capítulo 2 — e também PyPy e GraalPy, as implementações alternativas do Capítulo 1. Criar um ambiente com uma versão específica é uma linha:

```
uv venv --python 3.13 .venv
```

Se a versão pedida não estiver em disco, o **uv** baixa o interpretador e segue. Ele não pede `sudo`, não escreve em `/usr` e não conhece o `EXTERNALLY-MANAGED` porque nunca chega perto do Python do sistema.

pyenv é a alternativa mais antiga e ainda muito usada. A diferença de filosofia importa: o **pyenv** **compila** o interpretador na sua máquina, o que exige as bibliotecas de desenvolvimento instaladas e leva alguns minutos, mas produz um binário adaptado ao seu sistema. Ele também oferece um recurso que o **uv** trata de outra forma: o arquivo `.python-version` no diretório do projeto, que faz o **pyenv** trocar de versão automaticamente quando você entra na pasta.

Os pacotes da distribuição resolvem o caso em que a versão que você quer está no repositório. No Debian e no Kali, versões adicionais aparecem como pacotes próprios quando existem, e a instalação é a de sempre. No Ubuntu, o repositório extra **deadsnakes** é a fonte tradicional de versões que não vieram no sistema. Vale a advertência: instalar um `python3.12` pelo `apt` ao lado do `python3.13` do sistema é seguro, porque são pacotes distintos com arquivos distintos; o que não é seguro é usar `pip` com privilégio em qualquer um dos dois.

O instalador oficial de `python.org` é a rota natural no Windows e no macOS, e a única que este manual recomenda no Windows para quem não vai usar `uv`.

3.5. Windows e macOS em duas páginas

No **Windows**, o instalador oficial traz um programa que não existe no Linux e que resolve o problema de conviver com várias versões: o *launcher*, chamado `py`. Ele lê as instalações registradas e escolhe uma:

```
py -0p
```

```
-V:3.14 *          C:\Python314\python.exe
-V:Astral/CPython3.13.15
↪ C:\Users\Usuario\AppData\Roaming\uv\python\cpython-3.13.15-windows-x86_64-none\python.exe
```

O asterisco marca o padrão. `py -3.13` chama aquela versão específica, `py -3` chama o 3 mais recente, e `py -0p` lista tudo com o caminho. Como o `py` sabe de todas, ele é a forma mais confiável de escolher o interpretador no Windows — mais do que mexer no `PATH`.

Duas armadilhas específicas do Windows valem aviso. A primeira: se `python` abrir a Microsoft Store em vez de rodar, você encontrou o *stub* que a Microsoft instala por padrão. Ele não é um Python; é um atalho para a loja. A solução é usar `py -3` ou instalar de verdade e marcar a opção “**Add python.exe to PATH**” no instalador. A segunda: dentro de um ambiente virtual, prefira sempre `python -m pip` a `pip` solto, porque um `pip.exe` velho em outro diretório do `PATH` pode vencer a disputa e instalar no interpretador errado.

No **macOS**, o Python que vem com o sistema é ainda mais restrito do que no Linux — a Apple o marca como componente interno e já anunciou sua remoção. Use o instalador oficial ou o Homebrew (`brew install python@3.13`), e note que o Homebrew também aplica a PEP 668: `pip install` fora de um `venv` falha do mesmo jeito.

3.6. Jeito Pythônico

A convenção mais estabelecida do ecossistema é curta: **um ambiente virtual por projeto, dentro do projeto, chamado `.venv`, fora do controle de versão.**

```
# pythônico
cd ~/projetos/relatorios
python3 -m venv .venv
source .venv/bin/activate
python -m pip install requests
echo ".venv/" >> .gitignore
```

3. Instalando o Python: versões, gerenciadores e o Python do sistema

O nome `.venv` é reconhecido automaticamente pelo VS Code, pelo PyCharm, pelo `uv` e por várias outras ferramentas — usar outro nome só cria trabalho. E `.venv/` entra no `.gitignore` sempre: o ambiente é derivado, tem caminhos absolutos dentro dele e não é portátil entre máquinas. O que se versiona é a **lista** de dependências, tema do Volume 5.

O antipadrão a evitar tem várias formas, todas com a mesma raiz:

```
# antipadrão: escrever no Python do sistema
sudo pip install requests
pip install --break-system-packages requests
sudo python3 -m pip install --upgrade pip

# antipadrão: um ambiente só para tudo
source ~/.venv-geral/bin/activate # usado em quinze projetos diferentes
```

Os três primeiros brigam com o `apt` pelos mesmos arquivos. O quarto não quebra o sistema, mas devolve o problema que o `venv` resolvia: dois projetos que precisam de versões diferentes da mesma biblioteca voltam a se atropelar, e você descobre isso no pior momento.

Sobre `--break-system-packages`, uma nota honesta: ele existe para casos legítimos, quase todos em contêiner. Numa imagem Docker cujo único propósito é rodar um programa Python, não há conflito possível com ferramentas de distribuição, e a alternativa (criar um `venv` dentro da imagem) só acrescenta uma camada. Fora desse contexto, a opção faz exatamente o que o nome diz.

Para **instalar aplicativos** escritos em Python — `ruff`, `httpie`, `yt-dlp`, coisas que você chama pelo nome no terminal e não importa no seu código — nem `pip` nem `venv` manual são a resposta certa. O `pipx` cria um ambiente isolado por aplicativo e coloca só o executável no seu `PATH`:

```
sudo apt install pipx
pipx install ruff
```

O `uv` faz a mesma coisa com `uv tool install`. A regra mental é limpa: **biblioteca que você importa vai no `venv` do projeto; programa que você executa vai no `pipx`.**

Sobre versões, o que muda por aqui:

- **PEP 668** — o marcador `EXTERNALLY-MANAGED`. Chegou às distribuições ao longo de 2023; Debian 12, Ubuntu 23.04 e o Kali de então em diante. É a razão de tutoriais antigos com `sudo pip install` não funcionarem mais.
- **PEP 405** — o módulo `venv` na biblioteca padrão, desde o Python 3.3. Antes dele, `virtualenv` era um pacote externo obrigatório; hoje é opcional e serve sobretudo a quem precisa de compatibilidade mais ampla.
- **Python 3.11** — `python3 -m venv` passou a ser bem mais rápido, e `--upgrade-deps` ficou prático para já criar o ambiente com `pip` atualizado.

3.7. Laboratório

Objetivo: reproduzir a barreira da PEP 668, montar um ambiente virtual, provar o isolamento nos dois sentidos e inspecionar o que a ativação faz. Nada é instalado no sistema, e nada exige `sudo`.

O roteiro está em Linux (Kali/Debian). As diferenças de Windows estão no passo 9.

1. Confirme quem é o Python do sistema.

```
python3 --version
which -a python3
```

```
Python 3.13.12
/usr/bin/python3
/bin/python3
```

`/bin` é um link para `/usr/bin` nas distribuições atuais, então os dois caminhos são o mesmo arquivo.

2. Veja quem depende dele. Não pule este passo — é o que dá peso ao resto:

```
dpkg -l 'python3-*' | grep -c '^ii'
apt-cache rdepends --installed python3 | head -8
```

O primeiro comando conta os pacotes Python que o `apt` administra (529 na máquina deste capítulo; na sua, outro número). O segundo mostra alguns dos programas que quebrariam se o interpretador mudasse por baixo deles.

3. Provoque a PEP 668. Este comando **falha de propósito** e não altera nada no sistema:

```
pip install requests
```

Você deve ver o `error: externally-managed-environment` da abertura do capítulo, com o texto que a sua distribuição escreveu. **Não** execute a sugestão do `--break-system-packages`.

4. Encontre o marcador.

```
ls -l /usr/lib/python3.13/EXTERNALLY-MANAGED
head -4 /usr/lib/python3.13/EXTERNALLY-MANAGED
```

Compare o campo `Error` com a mensagem que o `pip` imprimiu no passo 3. É a mesma.

5. Crie o ambiente e leia o `pyvenv.cfg`.

```
mkdir -p ~/lab03 && cd ~/lab03
python3 -m venv .venv
cat .venv/pyvenv.cfg
```

Se der erro dizendo que `ensurepip` não está disponível, falta o pacote `python3-venv` (ou `python3-full`): instale com `sudo apt install python3-full` e repita. É a única vez neste laboratório em que `sudo` aparece, e é para instalar um pacote da distribuição pelo `apt` — não para mexer em pacote Python.

6. Compare os prefixos.

3. Instalando o Python: versões, gerenciadores e o Python do sistema

```
.venv/bin/python -c "import sys; print(sys.prefix); print(sys.base_prefix)"
```

Os dois têm de ser diferentes. Se forem iguais, você chamou o Python do sistema por engano.

7. Prove o isolamento nos dois sentidos.

```
.venv/bin/python -m pip install requests cowsay  
.venv/bin/python -c "import cowsay; print(cowsay.__file__)"  
python3 -c "import cowsay"
```

O ambiente encontra o `cowsay`; o sistema responde `ModuleNotFoundError`. Agora o caso mais interessante:

```
python3 -c "import requests; print(requests.__version__)"  
.venv/bin/python -c "import requests; print(requests.__version__)"
```

Dois números diferentes, dois arquivos diferentes, nenhum conflito. Se o seu Kali não trouxer `requests` do `apt`, o primeiro comando dá `ModuleNotFoundError` — o que também serve, só é menos ilustrativo.

8. Observe a ativação.

```
which python3  
source .venv/bin/activate  
which python  
echo "$VIRTUAL_ENV"  
deactivate  
which python3
```

A variável `VIRTUAL_ENV` é definida pela ativação e apagada pela desativação; é o que o seu prompt e o seu editor leem para mostrar qual ambiente está em uso. Confirme que, depois do `deactivate`, `which python3` volta a apontar para `/usr/bin/python3`.

9. No Windows. Os passos 3 e 4 não se aplicam — o Python oficial do Windows não é gerenciado externamente e o `pip` instala sem reclamar, o que é justamente por que ali o cuidado com ambientes é responsabilidade sua. O equivalente aos passos 1, 5 e 8:

```
py -Op  
py -3 -m venv .venv  
.\.venv\Scripts\Activate.ps1  
python -m pip install requests cowsay  
deactivate
```

Se o `Activate.ps1` for bloqueado pela política de execução do PowerShell, a saída sem baixar a guarda do sistema é chamar o interpretador pelo caminho: `.\.venv\Scripts\python.exe -m pip install ...` funciona sem ativação alguma.

10. Descarte e recrie. Para fechar, apague o ambiente e faça de novo:


```
rm -rf .venv
python3 -m venv .venv
```

Levou poucos segundos e nada foi perdido — porque não havia nada de valor lá dentro. Essa é a propriedade mais útil de um ambiente virtual: ele é descartável. Quando algo der errado nas dependências de um projeto, apagar o `.venv` e recriar é uma solução legítima, não uma gambiarra.

3.8. Resumo

- O Python de `/usr/bin` é infraestrutura da distribuição, não ferramenta de desenvolvimento. No Kali deste capítulo, 529 pacotes do `apt` e ferramentas como `theHarvester` e `weevely` dependem dele.
- A PEP 668 formalizou o arquivo `EXTERNALLY-MANAGED`, que faz o `pip` recusar instalação no interpretador do sistema. Não é bug: é a distribuição declarando que aquele Python tem dono.
- Um ambiente virtual é um `pyvenv.cfg`, um `bin/` e um `site-packages/` vazio. Ele não copia o interpretador: substitui o caminho de busca de pacotes, deixando a biblioteca padrão compartilhada e trocando `dist-packages` pelo `site-packages` do projeto.
- `sys.prefix` diferente de `sys.base_prefix` é a maneira canônica de saber que se está num `venv`. `activate` apenas insere um diretório no início do `PATH` e define `VIRTUAL_ENV`; `sudo` não herda nada disso.
- Para outra **versão** do interpretador, use `uv` (baixa binários prontos) ou `pyenv` (compila), ambos dentro do seu diretório pessoal. No Windows, o launcher `py` escolhe entre as versões instaladas.
- Convenção: um `.venv` por projeto, no projeto, no `.gitignore`. Biblioteca que se importa vai no `venv`; programa que se executa vai no `pipx`. `sudo pip install` não tem caso de uso legítimo numa máquina de trabalho.

4. O REPL, os scripts e o primeiro programa

Um cliente pede um orçamento: sete horas e meia de trabalho, a oitenta e cinco reais a hora. Você abre a calculadora do sistema, digita, anota o resultado. Meia hora depois vem outro cliente, e outro depois dele. No fim do dia você fez a mesma conta onze vezes e errou uma.

Existem duas ferramentas para isso, e a diferença entre elas é o assunto deste capítulo. Uma serve para **descobrir** qual é a conta; a outra, para **nunca mais** ter que fazê-la. Quem usa só a primeira repete trabalho para sempre. Quem usa só a segunda escreve arquivo para responder perguntas de dez segundos.

4.1. O REPL: onde se descobre

Digite `python3` no terminal, sem argumento nenhum:

```
Python 3.13.12 (main, Feb  4 2026, 15:06:39) [GCC 15.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> 2 + 3
5
>>> _ * 10
50
>>> horas = 7.5
>>> valor = 85
>>> horas * valor
637.5
>>> nome = "manual"
>>> nome.upper()
'MANUAL'
>>> len(nome)
6
>>> type(horas)
<class 'float'>
>>> for letra in "py":
...     print(letra, letra.upper())
...
p P
y Y
>>>
```

O nome **REPL** descreve o que aquele `>>>` faz, e está em Figura 4.1: *Read, Eval, Print, Loop* — lê a linha, avalia, imprime o resultado, volta a esperar.

Quatro detalhes dessa sessão valem atenção, porque são a diferença entre usar o REPL e brigar com ele.

4. O REPL, os scripts e o primeiro programa

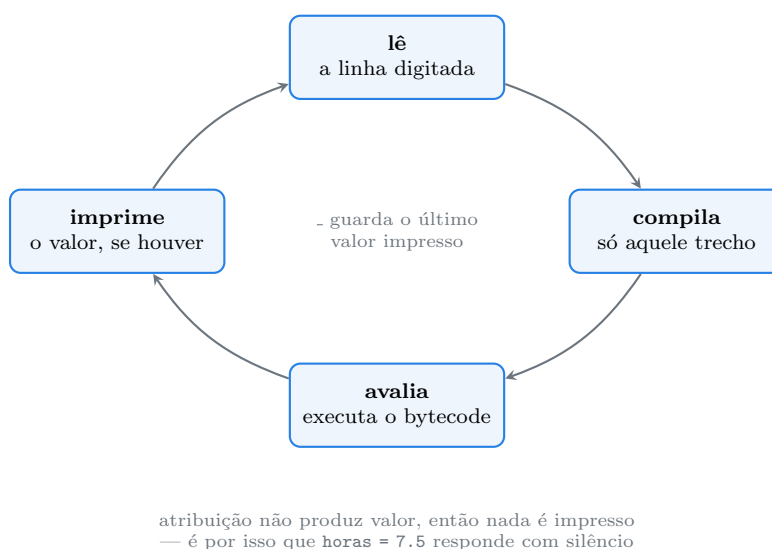


Figura 4.1.: O ciclo do REPL. A etapa de compilação é a mesma do Capítulo 2, só que sobre uma linha em vez de um arquivo.

Expressão imprime, atribuição não. `horas * valor` respondeu 637.5; `horas = 7.5` não respondeu nada. Não é economia de tinta: atribuição é um comando e não produz valor, então não há o que imprimir. Quando você digita algo e o REPL fica calado, ele está dizendo “isto não é uma expressão”.

O `_` guarda o último resultado. `2 + 3` deu 5, e `_ * 10` deu 50 sem que eu tivesse dado nome a nada. É útil para encadear exploração — calcule, veja, use o resultado no passo seguinte. E é uma armadilha em script: `_` só existe no modo interativo, e por convenção significa outra coisa em código (o valor que se descarta).

A resposta mostra o repr, não o texto. `nome.upper()` respondeu `'MANUAL'` com aspas, porque o REPL exibe a *representação* do objeto — a forma que você digitaria para recriá-lo. Já `print` mostra o texto sem aspas. É a razão de `<class 'float'>` aparecer com esse formato em `type(horas)`.

Bloco pede linha em branco. Quando você digita `for letra in "py":` e aperta Enter, o prompt muda para `...`, indicando que o comando não terminou. Para fechar o bloco, uma linha vazia. Esquecer isso e ficar preso no `...` é a primeira frustração de quase todo mundo.

4.1.1. O REPL do 3.13

O modo interativo foi reescrito no Python 3.13, e a diferença é grande o suficiente para valer nota. O REPL novo tem histórico que atravessa comandos de várias linhas, colagem de bloco inteiro sem que a indentação automática estrague o texto, prompt colorido e — o que mais economiza irritação — comandos que funcionam **sem parênteses**.

Isso não é um detalhe cosmético; é uma lista fixa dentro da implementação. Ela está em `_pyrepl/simple_interact.py`, no interpretador que você tem instalado:

```
grep -n -A 7 "^REPL_COMMANDS" /usr/lib/python3.13/_pyrepl/simple_interact.py
```

```

75:REPL_COMMANDS = {
76-     "exit": _sitebuiltins.Quitter('exit', ''),
77-     "quit": _sitebuiltins.Quitter('quit', ''),
78-     "copyright": _sitebuiltins._Printer('copyright', sys.copyright),
79-     "help": _sitebuiltins._Helper(),
80-     "clear": _clear_screen,
81-     "\x1a": _sitebuiltins.Quitter('\x1a', ''),
82-}

```

São cinco nomes — `exit`, `quit`, `copyright`, `help`, `clear` — mais o caractere de fim de arquivo do Windows. Antes do 3.13, digitar `exit` sozinho respondia com a dica irritante `Use exit() or Ctrl-D (i.e. EOF) to exit`, e você tinha que digitar de novo com os parênteses. Se quiser ver o comportamento antigo, force o REPL básico:

```
PYTHON_BASIC_REPL=1 python3
```

```

>>> exit
Use exit() or Ctrl-D (i.e. EOF) to exit
>>>

```

Uma sutileza que a implementação revela: a interceptação só vale se o nome **não** for uma variável sua. A linha que decide é `if statement in console.locals or statement not in REPL_COMMANDS`, ou seja, o seu `exit = 3` vence o comando. Faz sentido e evita surpresa.

Para pedir informação sem sair do REPL, dois embutidos resolvem quase tudo. O `help` mostra a documentação:

```

from orcamento import total
help(total)

```

```
Help on function total in module orcamento:
```

```

total(horas: float, valor_hora: float) -> float
    Devolve o valor total de `horas` trabalhadas a `valor_hora` por hora.

```

Note que o `help` mostrou a assinatura com as anotações de tipo e a primeira linha da documentação da função. E o `dir` lista o que um objeto sabe fazer:

```
print([m for m in dir("py") if not m.startswith("_")][:12])
```

```

['capitalize', 'casefold', 'center', 'count', 'encode', 'endswith',
↪  'expandtabs', 'find', 'format', 'format_map', 'index', 'isalnum']

```

Filtrei os nomes que começam com underscore para deixar visíveis os métodos de uso comum — os outros são o protocolo interno, assunto do Volume 8. Este par, `dir` para descobrir o que existe e `help` para descobrir como se usa, resolve a maior parte das dúvidas sem abrir o navegador.

4.2. O script: onde se repete

O REPL é excelente para descobrir e péssimo para guardar: feche o terminal e tudo se perde. O orçamento do cliente é uma tarefa recorrente, então ele merece um arquivo. Este é o primeiro programa completo do manual, `orcamento.py`:

```
#!/usr/bin/env python3
"""Calcula o valor de um serviço a partir de horas e valor por hora."""

import sys

def total(horas: float, valor_hora: float) -> float:
    """Devolve o valor total de `horas` trabalhadas a `valor_hora` por hora."""
    return horas * valor_hora

def main() -> int:
    if len(sys.argv) != 3:
        print(f"uso: {sys.argv[0]} HORAS VALOR_HORA", file=sys.stderr)
        return 2
    horas = float(sys.argv[1])
    valor_hora = float(sys.argv[2])
    print(f"{horas} h x R$ {valor_hora:.2f} = R$ {total(horas,
        ↪ valor_hora):.2f}")
    return 0

if __name__ == "__main__":
    sys.exit(main())
```

```
python3 orcamento.py 7.5 85
```

```
7.5 h x R$ 85.00 = R$ 637.50
```

São vinte linhas e cada bloco tem uma razão de ser. Vamos por partes, porque este esqueleto vai reaparecer em todo programa de linha de comando deste manual.

A primeira linha, o *shebang*. `#!/usr/bin/env python3` só tem efeito em sistemas Unix e só quando o arquivo é executável. Ela diz ao sistema operacional qual interpretador usar:

```
chmod +x orcamento.py
./orcamento.py 3 120
```

```
3.0 h x R$ 120.00 = R$ 360.00
```

Repare que se escreve `#!/usr/bin/env python3` e não `#!/usr/bin/python3`. A forma com `env` procura o `python3` no `PATH`, o que faz o script respeitar o ambiente virtual ativo — exatamente

o mecanismo do Capítulo 3. A forma com caminho fixo ignora o `venv` e vai sempre no Python do sistema, que quase nunca é o que você quer.

A segunda linha, a *docstring* do módulo. Um texto entre três aspas no começo do arquivo não é comentário: é um valor, guardado no atributo `__doc__` e legível por ferramentas.

```
python3 -c "import orcamento; print(orcamento.__doc__)"
```

```
Calcula o valor de um servico a partir de horas e valor por hora.
```

`sys.argv`, os argumentos da linha de comando. É uma lista de textos em que a posição zero é o nome do programa e o resto são os argumentos. São **textos**, sempre: `float(sys.argv[1])` é obrigatório, e é por isso que passar `abc` no lugar de um número derruba o programa com `ValueError`. Tratar isso bem é assunto do Capítulo 7 e, com ferramenta apropriada, do Volume 12.

Erro vai para `stderr`, e o programa devolve um código. Quando faltam argumentos, a mensagem sai por `sys.stderr` e o programa termina com 2:

```
python3 orcamento.py 2>/dev/null      # so o stdout
python3 orcamento.py 2>&1 >/dev/null  # so o stderr
python3 orcamento.py; echo $?
```

```
uso: orcamento.py HORAS VALOR_HORA
2
```

O primeiro comando não imprimiu nada — o `stdout` estava vazio. O segundo mostrou a mensagem de uso. Essa separação é o que permite `python3 orcamento.py 7.5 85 > planilha.txt` gravar só o resultado, com os erros continuando visíveis na tela. E o código de saída — 0 para sucesso, diferente de zero para falha — é como o shell sabe se deu certo, o que faz `&&` e `||` funcionarem.

Funções em vez de código solto. `total` faz a conta e `main` cuida da conversa com o mundo. A separação parece exagero em vinte linhas, mas é o que permite reaproveitar a conta sem executar o programa:

```
python3 -c "from orcamento import total; print(total(2, 50))"
```

```
100
```

Isso funciona por causa da última peça, que merece seção própria.

4.3. `if __name__ == "__main__"`

Todo módulo Python tem uma variável `__name__`. O valor dela depende de **como** o arquivo foi carregado, e é a única diferença entre “programa” e “biblioteca” na linguagem. Dois arquivos mostram isso. Primeiro `quemsou.py`:

4. O REPL, os scripts e o primeiro programa

```
print("modulo quemsou, __name__ =", __name__)

if __name__ == "__main__":
    print("  -> rodando como programa principal")
else:
    print("  -> fui importado por outro modulo")
```

E `usa.py`, que só o importa:

```
import quemsou

print("modulo usa, __name__ =", __name__)
```

Rodando o primeiro direto:

```
python3 quemsou.py
```

```
modulo quemsou, __name__ = __main__
-> rodando como programa principal
```

Rodando o segundo, que importa o primeiro:

```
python3 usa.py
```

```
modulo quemsou, __name__ = quemsou
-> fui importado por outro modulo
modulo usa, __name__ = __main__
```

Três coisas de uma vez. O módulo importado recebeu o próprio nome, `quemsou`. O arquivo que foi chamado na linha de comando recebeu `__main__` — e note que agora é o `usa.py` que tem esse valor. E o `print` do topo do `quemsou.py` **executou durante o import**: importar um módulo roda o código do nível superior dele, inteiro.

Esse último ponto é a razão de o guarda existir. Figura 4.2 mostra os dois caminhos.

Sem aquele `if`, o `from orcamento import total` do exemplo anterior teria rodado o `main()`, que olharia o `sys.argv` do processo que importou, não acharia dois argumentos e mataria o programa com código 2. Com o guarda, o mesmo arquivo é as duas coisas: um programa quando chamado e uma biblioteca quando importado.

Vale desfazer uma confusão comum: `if __name__ == "__main__"` não é uma declaração de “função principal” como o `main` de C. É uma condição normal, avaliada quando o módulo carrega, sobre uma variável normal. Não há nada de especial nela além da convenção — e a convenção é universal.

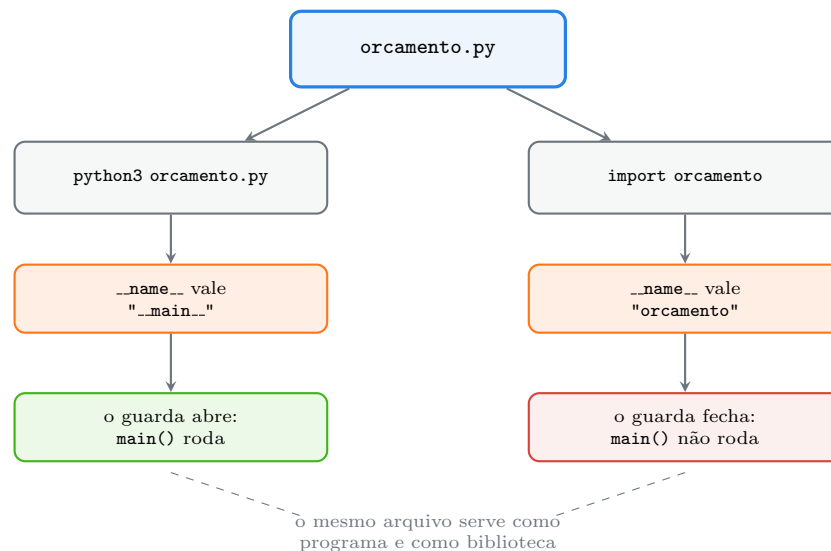


Figura 4.2.: Sem o guarda, importar o módulo executaria o programa.

4.4. As cinco formas de rodar código Python

Além do REPL e do script, existem outras. Cada uma tem um uso próprio:

Forma	Para quê
<code>python3</code>	REPL: explorar, testar ideia, inspecionar objeto
<code>python3 arquivo.py</code>	rodar um programa que você escreveu
<code>python3 -c "código"</code>	uma linha só, direto no shell ou num script de shell
<code>python3 -m modulo</code>	rodar um módulo instalado como se fosse programa
<code>python3 -i arquivo.py</code>	rodar o arquivo e cair no REPL com o estado dele

O `-m` merece destaque porque resolve um problema real. Ele procura o módulo no caminho de importação e o executa com `__name__` valendo `__main__`:

```
python3 -m quemsou
```

```
modulo quemsou, __name__ = __main__
-> rodando como programa principal
```

Muitos módulos da biblioteca padrão são úteis assim, sem que você escreva nada:

```
python3 -m json.tool <<< '{"cliente":"sitio","quedas":4}'
```

4. O REPL, os scripts e o primeiro programa

```
{
    "cliente": "sitio",
    "quedas": 4
}
```

O `-m` também é a forma correta de chamar o `pip` (`python3 -m pip`, como no Capítulo 3), porque garante que o `pip` executado é o do interpretador que você invocou, e não outro que esteja no `PATH`.

Cuidado com uma diferença sutil no `-c`: o `sys.argv` começa com a string `'-c'`, não com um nome de arquivo:

```
python3 -c "import sys; print(sys.argv)" um dois tres
```

```
['-c', 'um', 'dois', 'tres']
```

Já o `-i` é a ponte entre os dois mundos deste capítulo, e a ferramenta de depuração mais subestimada que existe. Ele roda o arquivo e, em vez de terminar, abre o REPL com tudo que o arquivo definiu ainda vivo — inclusive quando o arquivo morreu com uma exceção. Você cai no prompt com as variáveis no estado em que estavam e pode olhar uma por uma.

4.5. Jeito Pythônico

O esqueleto de `orcamento.py` é a forma canônica de escrever um programa de linha de comando em Python, e cada peça dele tem um antipadrão correspondente.

```
# antipadrão: código solto no nível do módulo
import sys

horas = float(sys.argv[1])
valor = float(sys.argv[2])
print(horas * valor)
```

```
# pythônico: função + guarda
def main() -> int:
    ...
    return 0

if __name__ == "__main__":
    sys.exit(main())
```

A versão de cima funciona e é o que quase todo mundo escreve primeiro. Ela tem três problemas que só aparecem depois: não pode ser importada sem executar, não pode ser testada (o Volume 11 vai querer chamar `total` num teste), e não tem como devolver código de saída diferente de zero sem espalhar `sys.exit` pelo meio.

Sobre o guarda, a forma idiomática é `sys.exit(main())` e não só `main()`. Assim o valor devolvido pela função vira o código de saída do processo, e você escreve um único `return` em vez de chamar `sys.exit` em cada ramo.

Três outros pontos de estilo:

Mensagem de erro vai para `stderr`. `print(..., file=sys.stderr)` e não `print(...)`. Quem usa o seu programa num pipe agradece.

`_` não sai do REPL. No modo interativo ele é o último resultado; em código, a convenção é o oposto — `_` marca o valor que você está descartando de propósito, como em `for _ in range(3):` quando o índice não importa.

Duas aspas triplas no topo, não um comentário. Docstring é dado, comentário é ruído para a máquina. Ferramentas leem `__doc__`; nenhuma lê o `#`.

Sobre versões:

- **Python 3.13** — REPL novo (PEP 762), com `exit`, `quit`, `help`, `clear` e `copyright` sem parênteses, histórico multilinha e colagem de bloco. `PYTHON_BASIC_REPL=1` volta ao antigo.
- **Python 3.11** — `-P` desliga a inclusão automática do diretório do script no caminho de importação, e `PYTHONSAFEPATH` faz o mesmo por variável de ambiente. Interessa a quem já foi mordido por um arquivo local chamado `json.py` sombreando o módulo da biblioteca padrão.
- **Python 3.12** — o `sys.last_exc` guarda a última exceção no modo interativo, mais conveniente que os três atributos antigos.

Por fim, a divisão de trabalho que resume o capítulo: **use o REPL para perguntas e o arquivo para respostas**. Se você digitou a mesma coisa no REPL duas vezes, é hora de abrir um arquivo. Se você está editando um arquivo para descobrir o nome de um método, é hora de abrir o REPL.

4.6. Laboratório

Objetivo: usar o REPL para descobrir, transformar a descoberta num programa completo e verificar as duas identidades do mesmo arquivo. Só biblioteca padrão.

1. Ambiente.

```
mkdir -p ~/lab04 && cd ~/lab04
python3 -m venv .venv && source .venv/bin/activate
python --version
```

Python 3.13.12

2. Explore no REPL. Abra `python` e reproduza a sessão da abertura do capítulo. Depois experimente três coisas fora do roteiro:

```
>>> 10 / 3
>>> 10 // 3
>>> 10 % 3
```

4. O REPL, os scripts e o primeiro programa

Anote os três resultados. Divisão em Python devolve `float` mesmo entre inteiros — e isso derruba quem chega de C. O Volume 2 trata do assunto.

3. Confirme que atribuição não imprime. Digite `x = 5` e depois `x`. A primeira linha responde com silêncio, a segunda com 5. Agora digite `x == 5`: isso é uma expressão, e responde `True`.

4. Use `dir` e `help` para descobrir sozinho. Sem consultar nada externo, descubra o que faz o método `zfill` de um texto:

```
>>> help("".zfill)
>>> "42".zfill(5)
```

Este é o hábito a construir: a documentação está dentro do interpretador.

5. Teste os comandos sem parênteses. Digite `help` sozinho e aperte Enter — no 3.13 entra no modo de ajuda interativo (saia com Enter em branco). Depois compare com o REPL antigo:

```
PYTHON_BASIC_REPL=1 python
```

Digite `exit` sem parênteses. Você deve ver `Use exit() or Ctrl-D (i.e. EOF) to exit`. Saia com `exit()` e reabra o `python` normal, onde `exit` sozinho funciona.

6. Escreva o programa. Crie `orcamento.py` com o conteúdo mostrado no capítulo e rode:

```
python orcamento.py 7.5 85
```

```
7.5 h x R$ 85.00 = R$ 637.50
```

7. Exercite as bordas. Três execuções que **falham de propósito**, para você ver o que cada uma faz:

```
python orcamento.py           # faltam argumentos
python orcamento.py 7.5       # falta um
python orcamento.py 7.5 abc   # o segundo não é número
```

As duas primeiras devem imprimir a mensagem de uso e sair com código 2 — confirme com `echo $?`. A terceira estoura um `ValueError` com traceback, porque `float("abc")` não tem resposta. Guarde essa diferença: erro previsto é mensagem, erro imprevisto é traceback. O Capítulo 7 é sobre isso.

8. Separe os fluxos.

```
python orcamento.py 7.5 85 > saida.txt
cat saida.txt
python orcamento.py > vazio.txt
cat vazio.txt
```

O primeiro `cat` mostra o resultado; o segundo mostra um arquivo vazio, porque a mensagem de uso foi para `stderr` e continuou aparecendo na tela.

9. Torne executável. Em Linux ou macOS:

```
chmod +x orcamento.py
./orcamento.py 3 120
```

```
3.0 h x R$ 120.00 = R$ 360.00
```

No Windows não há `chmod`; o equivalente é a associação de extensão feita pelo instalador, e `py orcamento.py 3 120` sempre funciona.

10. Prove as duas identidades. Crie `quemsou.py` e `usa.py` como no capítulo e rode os dois, mais o `-m`:

```
python quemsou.py
python usa.py
python -m quemsou
```

Confirme que `__name__` vale `__main__` no primeiro e no terceiro, e `quemsou` no segundo.

11. Importe sem executar.

```
python -c "from orcamento import total; print(total(2, 50))"
```

```
100
```

Agora **apague** as duas últimas linhas do `orcamento.py` (o `if` e o `sys.exit`), acrescente `main()` solto no lugar e repita o comando acima. Você deve ver a mensagem de uso e o programa morrer — porque o `main` rodou durante o `import` e não achou os argumentos. Desfaça a mudança. Este é o passo que faz o guarda parar de ser superstição.

12. Caia no REPL com o estado.

```
python -i orcamento.py 7.5 85
```

Não se assuste com o que aparece antes do prompt:

```
File "/tmp/lab04/orcamento.py", line 23, in <module>
    sys.exit(main())
    ~~~~~~^~~~~~
SystemExit: 0
```

Isso **não** é um erro. O `sys.exit` funciona levantando a exceção `SystemExit`, e normalmente o interpretador a intercepta em silêncio para encerrar o processo. Com `-i`, como o processo não vai terminar, ela é exibida como qualquer outra exceção — e o 0 no fim é o código de saída de sucesso. No seu terminal o traceback sai colorido, novidade do 3.13.

Já no prompt, tudo do arquivo está carregado. Experimente:

4. O REPL, os scripts e o primeiro programa

```
>>> total(1, 1)
1
>>> sys.argv
['orcamento.py', '7.5', '85']
>>> horas
```

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
    horas
NameError: name 'horas' is not defined
```

`total` e `sys` existem porque são do nível superior do módulo; `horas` não, porque era variável local do `main`. É exatamente a distinção de escopo que o Volume 3 vai formalizar. Saia com `exit`.

13. Use um módulo da biblioteca padrão como programa.

```
echo '{"cliente":"sitio","quedas":4}' | python -m json.tool
python -m this
python -m timeit "sum(range(100))"
```

14. Feche com `deactivate`.

4.7. Resumo

- O REPL é para descobrir; o arquivo é para repetir. Digitou a mesma coisa duas vezes no REPL: abra um arquivo.
- No REPL, expressão imprime e atribuição não; `_` guarda o último valor impresso; a resposta é o `repr` do objeto, não o texto; bloco só fecha com linha em branco.
- O REPL do 3.13 aceita `exit`, `quit`, `help`, `clear` e `copyright` sem parênteses — uma lista fixa em `_pyrepl/simple_interact.py`, que perde para uma variável sua de mesmo nome. `PYTHON_BASIC_REPL=1` volta ao comportamento antigo.
- Um programa de linha de comando tem shebang com `env`, docstring de módulo, a lógica em funções, `sys.argv` convertido explicitamente, erro em `sys.stderr` e código de saída via `sys.exit(main())`.
- `__name__` vale `"__main__"` no arquivo que foi chamado e o nome do módulo quando importado. O guarda `if __name__ == "__main__"` é o que permite ao mesmo arquivo ser programa e biblioteca — sem ele, importar executa.
- Cinco formas de rodar: REPL, arquivo, `-c` para uma linha, `-m` para módulo instalado (é assim que se chama o `pip`), `-i` para cair no REPL com o estado do arquivo.

5. Ambiente de trabalho: editor, VS Code, Jupyter e o terminal

O código roda no terminal. No editor, a mesma linha aparece sublinhada de amarelo com “import não pode ser resolvido”. Você roda de novo no terminal: funciona. Aperta o botão de executar do editor: `ModuleNotFoundError`. Abre um notebook para testar e lá o import funciona, mas a função que você acabou de corrigir no arquivo continua devolvendo o valor antigo.

Nada disso é bug de ferramenta. São três programas diferentes perguntando a três interpretadores diferentes, e a solução para todos os casos é a mesma. Este capítulo monta o ambiente de trabalho de forma que essa classe de problema deixe de existir.

5.1. Uma regra que resolve a maioria dos problemas

O ambiente virtual do Capítulo 3 é um interpretador com um conjunto de pacotes. O terminal sabe qual usar porque a ativação mexeu no `PATH`. O editor não vê o seu `PATH`, e o kernel de um notebook é um terceiro processo, iniciado por quem sabe qual Python. Figura 5.1 mostra o arranjo que funciona.

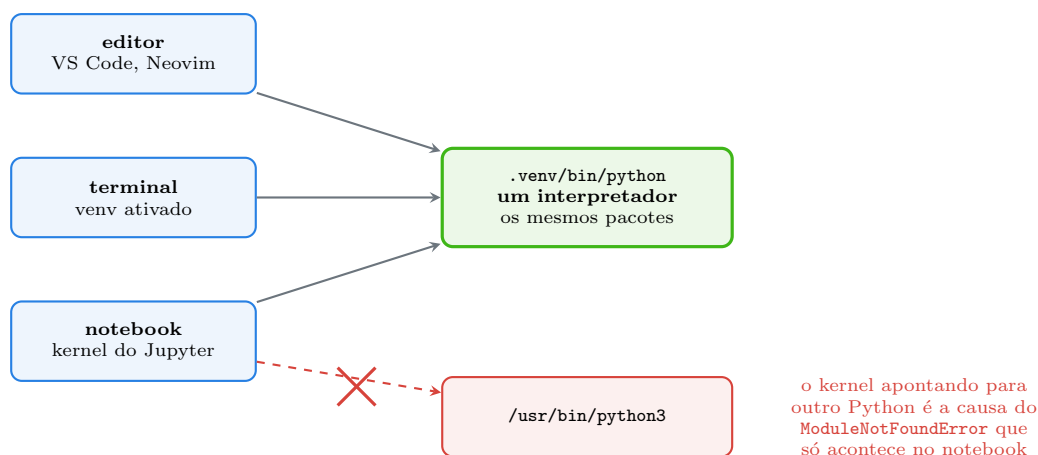


Figura 5.1.: Editor, terminal e notebook precisam apontar para o mesmo interpretador.

A regra é: **um ambiente virtual por projeto, e as três ferramentas apontando para ele**. Quando alguma coisa funciona num lugar e não no outro, a primeira pergunta não é “qual é o bug”, é *qual Python cada um está usando*. E a resposta é sempre a mesma linha, rodada de dentro de cada ferramenta:

```
import sys
print(sys.executable)
```

Se o terminal responde `/home/voce/projeto/.venv/bin/python` e o notebook responde `/usr/bin/python3`, você achou o problema em dez segundos, sem investigar mais nada.

5.2. O terminal

O terminal não é um detalhe de acompanhamento — é onde o Python é executado de verdade, em servidor, em contêiner e em automação. Vale saber o mínimo bem.

Quatro coisas cobrem o dia a dia. **Navegar:** `cd`, `ls`, `pwd`. **Ativar o ambiente:** `source .venv/bin/activate`, e conferir com `which python`. **Rodar:** `python arquivo.py`, `python -m modulo`, `python -c` — as formas do Capítulo 4. E **redirecionar:** `>` para gravar a saída, `2>` para gravar o erro, `|` para passar a saída de um programa como entrada de outro.

Esse último merece um exemplo, porque é o que torna um script Python uma peça reutilizável em vez de um programa isolado:

```
python orcamento.py 7.5 85 | tee ultimo-orcamento.txt
```

O programa não sabe que está num pipe, e é justamente por isso que funciona: ele escreveu no `stdout`, como manda a convenção do Capítulo 4, e o shell cuidou do resto.

Duas variáveis de ambiente valem conhecer agora. `PYTHONPATH` acrescenta diretórios ao caminho de busca de módulos — útil em emergência, e um cheiro de problema de estrutura quando aparece em uso permanente (o Volume 5 resolve isso direito). E `PYTHONDONTWRITEBYTECODE=1`, que desliga o `__pycache__` do Capítulo 2.

No **Windows**, o equivalente é o PowerShell, com `Activate.ps1` no lugar do `source`. O terminal integrado do VS Code herda a ativação, o que resolve metade dos problemas deste capítulo de graça.

5.3. O editor

Qualquer editor serve para escrever Python. O que muda a produtividade é ter cinco coisas funcionando: realce de sintaxe, autocompletar que conhece os seus objetos, o erro apontado antes de rodar, formatação automática ao salvar e um depurador. Este manual usa o **VS Code** nos exemplos por ser gratuito, multiplataforma e o mais comum — mas nada aqui depende dele.

O essencial da configuração é uma extensão e uma escolha. A extensão é a **Python**, da Microsoft, que traz junto o servidor de linguagem Pylance. A escolha é o interpretador: `Ctrl+Shift+P`, comando **Python: Select Interpreter**, e aponte para `./.venv/bin/python`. Isso é o que faz o sublinhado amarelo desaparecer, porque o editor passa a olhar os pacotes do ambiente certo.

Melhor do que escolher na mão é deixar a escolha registrada no projeto. Crie `.vscode/settings.json`:

```
{
  "python.defaultInterpreterPath": "${workspaceFolder}/.venv/bin/python",
  "editor.formatOnSave": true,
  "editor.rulers": [88],
  "files.trimTrailingWhitespace": true,
  "files.insertFinalNewline": true,
  "[python]": {
    "editor.defaultFormatter": "charliermarsh.ruff",
    "editor.codeActionsOnSave": {
      "source.organizeImports": "explicit"
    }
  }
}
```



```
}
}
```

Esse arquivo **vai para o Git**. É a diferença entre “funciona na minha máquina” e “funciona para quem clonar o repositório”: qualquer pessoa que abrir o projeto recebe o mesmo interpretador, a mesma largura de linha e a mesma formatação ao salvar. No Windows, o caminho do interpretador é `${workspaceFolder}/.venv/Scripts/python.exe`.

O 88 dos **rulers** não é arbitrário: é a largura de linha que o formatador usa por padrão, e a régua na tela evita a briga de reformatação a cada salvamento.

5.3.1. O depurador, que substitui o `print`

O recurso mais subutilizado do editor é o depurador. Colocar `print` para descobrir o valor de uma variável funciona, mas é lento: você edita, roda, lê, edita de novo, e no fim precisa lembrar de apagar tudo.

Com o depurador, você clica na margem esquerda para criar um ponto de parada, aperta F5 e o programa congela naquela linha com todas as variáveis visíveis num painel. Dá para avançar linha por linha, entrar dentro de uma função, e — o que mais importa — digitar expressões no console de depuração usando o estado real daquele instante.

O equivalente sem editor nenhum já está na linguagem, desde o Python 3.7:

```
def total(horas, valor_hora):
    breakpoint()
    return horas * valor_hora
```

`breakpoint()` para a execução e abre o `pdb`, o depurador de linha de comando da biblioteca padrão. É o mesmo recurso, disponível em servidor por SSH, onde não há editor gráfico. O Volume 11 tem um capítulo sobre isso; por ora, saiba que existe e que a variável `PYTHONBREAKPOINT=0` desliga todos os `breakpoint()` de uma vez, o que é útil quando você esqueceu um.

5.4. Linter e formatador: *ruff*

Duas ferramentas diferentes com nomes parecidos. O **formatador** arruma a aparência: espaços, quebras de linha, aspas. Ele não muda o que o programa faz e não tem opinião sobre a sua lógica. O **linter** procura problemas: `import` que não é usado, variável definida e nunca lida, nome fora da convenção, construção que tem uma forma melhor.

O **ruff** faz as duas coisas, é escrito em Rust e é rápido o bastante para rodar a cada salvamento. Instale no ambiente do projeto:

```
python -m pip install ruff
ruff --version
```

```
ruff 0.16.2
```

Vamos dar a ele um arquivo com problemas de propósito, `bagunca.py`:

5. Ambiente de trabalho: editor, VS Code, Jupyter e o terminal

```
import os
import sys
from pathlib import Path

def Media( valores ):
    total=0
    for i in range(len(valores)):
        total = total + valores[i]
    return total/len(valores)

x = Media([1,2,3])
print( "media:",x )
```

Primeiro o linter, com as regras que vêm ligadas por padrão:

```
ruff check --output-format concise bagunca.py
```

```
bagunca.py:1:1: I001 [*] Import block is un-sorted or un-formatted
bagunca.py:1:8: F401 [*] `os` imported but unused
bagunca.py:2:8: F401 [*] `sys` imported but unused
bagunca.py:3:21: F401 [*] `pathlib.Path` imported but unused
Found 4 errors.
[*] 4 fixable with the `--fix` option.
```

Cada linha traz arquivo, linha, coluna, código da regra e a explicação. O [*] marca o que a ferramenta sabe corrigir sozinha. As regras padrão são conservadoras; ligando mais famílias, aparece mais:

```
ruff check --output-format concise --select E,F,I,N,UP,B,SIM bagunca.py
```

```
bagunca.py:1:1: I001 [*] Import block is un-sorted or un-formatted
bagunca.py:1:8: F401 [*] `os` imported but unused
bagunca.py:2:8: F401 [*] `sys` imported but unused
bagunca.py:3:21: F401 [*] `pathlib.Path` imported but unused
bagunca.py:5:5: N802 Function name `Media` should be lowercase
Found 5 errors.
[*] 4 fixable with the `--fix` option.
```

Entrou o N802: `Media` deveria ser `media`, porque `CamelCase` é para classes. E note que ele **não** é corrigível automaticamente — renomear uma função exige saber quem a chama, e isso a ferramenta não decide por você.

Agora o formatador, mostrando o que mudaria antes de mudar:

```
ruff format --diff bagunca.py
```

```

--- bagunca.py
+++ bagunca.py
@@ -2,11 +2,13 @@
     import sys
     from pathlib import Path

-def Media( valores ):
-    total=0
+
+def Media(valores):
+    total = 0
+    for i in range(len(valores)):
+        total = total + valores[i]
-    return total/len(valores)
+    return total / len(valores)
+

-x = Media([1,2,3])
-print( "media:",x )
+x = Media([1, 2, 3])
+print("media:", x)

```

Espaços dentro dos parênteses removidos, espaço em volta dos operadores acrescentado, duas linhas em branco antes da função — tudo o que a PEP 8 pede e ninguém quer fazer à mão. Aplicando as duas ferramentas:

```

ruff check --select E,F,I,N,UP,B,SIM --fix --output-format concise bagunca.py
ruff format bagunca.py

```

```

bagunca.py:2:5: N802 Function name `Media` should be lowercase
Found 4 errors (3 fixed, 1 remaining).
1 file reformatted

```

O resultado:

```

def Media(valores):
    total = 0
    for i in range(len(valores)):
        total = total + valores[i]
    return total / len(valores)

x = Media([1, 2, 3])
print("media:", x)

```

Vale olhar com atenção para **o que continua errado**. Os três imports inúteis saíram, o espaçamento está correto, mas o laço percorrendo a lista por índice — o antipadrão do Capítulo 1 — passou intacto, e o nome `Media` só recebeu um aviso. Nenhuma ferramenta escreve `sum(valores) / len(valores)` no seu lugar.

5. Ambiente de trabalho: editor, VS Code, Jupyter e o terminal

Isso define o limite da automação, e é um limite honesto: **linter e formatador cuidam do que é mecânico para liberar a sua atenção para o que não é**. Eles não substituem saber a linguagem. Um arquivo aprovado pelo **ruff** pode ser um programa ruim.

A configuração do **ruff** mora no `pyproject.toml`, na raiz do projeto, e também vai para o Git:

```
[tool.ruff]
line-length = 88
target-version = "py313"

[tool.ruff.lint]
select = ["E", "F", "I", "N", "UP", "B", "SIM"]
```

Com isso, **ruff check** e **ruff format** já saem com as regras do projeto, sem `--select` na linha de comando, e o editor lê o mesmo arquivo. Uma fonte de verdade para você, para a equipe e para a integração contínua — assunto do Volume 16.

5.5. O notebook

Um notebook Jupyter é uma sequência de células que você executa na ordem que quiser, com o resultado — texto, tabela, gráfico — logo abaixo de cada uma. Instale no ambiente do projeto:

```
python -m pip install jupyterlab
jupyter --version
```

```
Selected Jupyter core packages...
IPython          : 9.16.1
ipykernel        : 7.3.0
jupyter_client   : 8.9.1
jupyter_core     : 5.9.1
jupyter_server   : 2.20.0
jupyterlab       : 4.6.3
nbclient         : 0.11.0
nbconvert        : 7.17.1
nbformat         : 5.11.0
```

Abra com **jupyter lab**, que sobe um servidor local e abre o navegador. O VS Code também abre arquivos `.ipynb` direto, o que costuma ser mais confortável.

Notebook é excelente para **explorar dados**: carregar um arquivo, olhar, filtrar, desenhar um gráfico, mudar de ideia. O ciclo é curto e o estado fica vivo entre as células, então você não recarrega o arquivo de 200 MB a cada tentativa. Os volumes de dados e visualização deste manual vivem nesse modo.

E notebook tem uma armadilha que precisa ser entendida antes do primeiro uso, não depois: **o estado depende da ordem em que você executou, não da ordem em que as células aparecem na tela**.

5.5.1. A ordem de execução

Imagine três células, nesta ordem no arquivo:

```
# célula A
taxa = 0.10
print("taxa definida:", taxa)
```

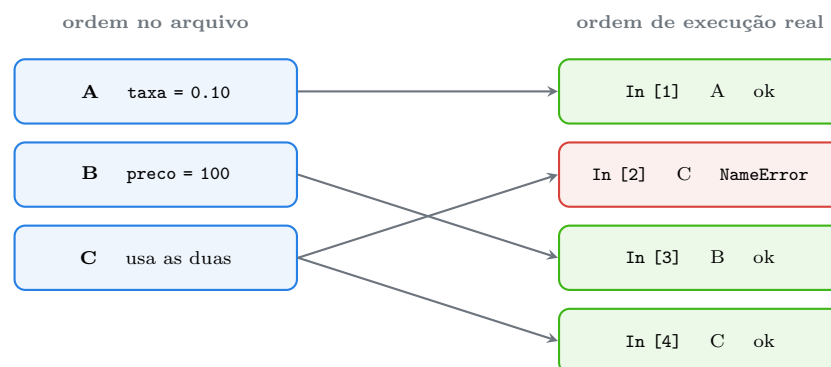
```
# célula B
preco = 100
print("preco definido:", preco)
```

```
# célula C
print("total:", preco * (1 + taxa))
```

Agora suponha que você clicou nelas na ordem A, C, B, C — o que é absolutamente normal quando se está explorando. Foi o que rodamos num kernel de verdade:

```
clique na celula A -> In [1]
    taxa definida: 0.1
clique na celula C -> In [2]
    NameError: name 'preco' is not defined
clique na celula B -> In [3]
    preco definido: 100
clique na celula C -> In [4]
    total: 110.00000000000001
```

O número entre colchetes é o **contador de execução**, e é a informação mais importante da tela. Ele não é o número da célula: é a ordem em que as coisas aconteceram. Figura 5.2 põe as duas ordens lado a lado.



o contador entre colchetes conta **execuções**, não células — um notebook cujos números não sejam 1, 2, 3... em sequência não roda igual para outra pessoa

Figura 5.2.: A mesma célula C falha e funciona, dependendo do que rodou antes.

Duas consequências práticas. A primeira: um notebook com contadores fora de ordem **não é reproduzível**. Ele funciona na sua tela porque o kernel guarda variáveis que não estão mais

5. Ambiente de trabalho: editor, VS Code, Jupyter e o terminal

em nenhuma célula, ou porque uma célula foi executada com um código que você depois editou. Passe para outra pessoa e não roda.

A segunda: o hábito que evita tudo isso é **Restart Kernel and Run All Cells** antes de considerar o trabalho pronto. Isso apaga o estado, executa tudo de cima para baixo e prova que o notebook é o que ele parece ser. Se falhar, era mentira.

Como brinde, aquele 110.000000000000001 no lugar de 110.0 não é erro do Jupyter: é o comportamento do ponto flutuante binário, que tem um capítulo inteiro reservado no Volume 2.

Notebook é ruim para outras coisas, e vale dizer quais. Código que vai para produção, biblioteca reutilizável, qualquer coisa que precise de teste automatizado ou de revisão em *pull request* — nada disso pertence a um `.ipynb`. O formato é JSON com a saída embutida, o que faz o `diff` do Git ficar ilegível e o histórico engordar com imagens. A divisão saudável é: **explore no notebook, mude para arquivo .py assim que o código tiver valor além daquela sessão.**

5.6. Outras opções

PyCharm é um ambiente completo, com o depurador mais confortável do ecossistema e refatoração de verdade. A edição Community é gratuita. Custa mais memória e tem mais curva de aprendizado.

Neovim com `pylsp` ou `pyright` entrega o mesmo autocompletar e diagnóstico dentro do terminal, o que importa em servidor remoto. Exige montar a configuração.

Spyder imita o Matlab e agrada quem vem da computação científica.

E `python -i` do Capítulo 4 continua sendo o ambiente mais rápido de todos para uma pergunta pequena.

5.7. Jeito Pythonico

A convenção sobre ambiente de trabalho é curta: **versione a configuração, nunca o ambiente.**

```
# vai para o Git
pyproject.toml      # regras do ruff, dependências, metadados
.vscode/settings.json # interpretador, formatação ao salvar
.gitignore

# não vai para o Git
.venv/              # derivado, com caminhos absolutos dentro
__pycache__/        # cache de bytecode
*.ipynb_checkpoints/ # rascunhos do Jupyter
```

O `.gitignore` mínimo de um projeto Python:

```
.venv/
__pycache__/
*.py[cod]
.ipynb_checkpoints/
.ruff_cache/
```

Três antipadrões comuns:

```
# antipadrão: instalar a ferramenta no ambiente do projeto sem precisar
python -m pip install ruff black isort flake8 pylint mypy

# pythônico: uma ferramenta que faz o trabalho de várias
python -m pip install ruff
```

O `ruff` substitui `flake8`, `isort`, `black` e boa parte do `pylint` com uma configuração só. Empilhar ferramentas com opiniões diferentes sobre a mesma linha gera conflito de formatação a cada salvamento.

```
# antipadrão: depurar por print e esquecer de apagar
print("AQUI 1", valores)
print(">>>", total)

# pythônico: ponto de parada
breakpoint()
```

```
# antipadrão: um notebook de 200 células que é o produto final
analise.ipynb

# pythônico: notebook para explorar, .py para o que ficou de pé
explorar.ipynb -> src/relatorio.py + tests/test_relatorio.py
```

Sobre versões e ferramentas:

- **Python 3.7** — `breakpoint()` embutido (PEP 553), com `PYTHONBREAKPOINT` para desligar ou trocar o depurador.
- **Python 3.11** — `PYTHONSAFEPATH` evita que um arquivo local com nome de módulo da biblioteca padrão sobre escreva o original.
- **ruff** — pede `target-version` no `pyproject.toml` para saber quais modernizações propor. Com `py313`, ele sugere sintaxe que não existia em versões anteriores; com `py39`, não.
- **PEP 8** é a referência de estilo, e a largura de linha de 88 caracteres é convenção de ferramenta, não da PEP (que fala em 79). O Volume 11 trata do assunto.

5.8. Laboratório

Objetivo: montar um projeto com ambiente, configuração versionada, linter, formatador e notebook, e provar que as três ferramentas usam o mesmo interpretador.

1. Estrutura e ambiente.

5. Ambiente de trabalho: editor, VS Code, Jupyter e o terminal

```
mkdir -p ~/lab05 && cd ~/lab05
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -c "import sys; print(sys.executable)"
```

A última linha tem de apontar para dentro de `~/lab05/.venv`. Guarde esse caminho: ele é o gabarito dos passos 7 e 9.

2. Instale as ferramentas.

```
python -m pip install ruff jupyterlab
ruff --version
```

```
ruff 0.16.2
```

A sua versão provavelmente será mais nova; o `ruff` muda rápido, e mensagens podem diferir em detalhe.

3. Crie o arquivo problemático. Salve `bagunca.py` com o conteúdo do capítulo — **exatamente** como está, com os espaços tortos.

4. Rode o linter.

```
ruff check --output-format concise bagunca.py
```

Quatro erros. Agora com mais regras:

```
ruff check --output-format concise --select E,F,I,N,UP,B,SIM bagunca.py
```

Cinco, com o N802 do nome `Media`.

5. Veja o formatador antes de aplicar.

```
ruff format --diff bagunca.py
```

Leia o diff com calma: `-` é o que sai, `+` é o que entra. Este hábito — olhar antes de deixar a ferramenta escrever — vale para o resto da vida profissional.

6. Aplique e confira o que sobrou.

```
ruff check --select E,F,I,N,UP,B,SIM --fix --output-format concise bagunca.py
ruff format bagunca.py
cat bagunca.py
```

Confirme que o laço por índice continua lá. Agora **corrija à mão** o que a ferramenta não corrige:


```
def media(valores):
    return sum(valores) / len(valores)

x = media([1, 2, 3])
print("media:", x)
```

7. Versione a configuração. Crie `pyproject.toml` e `.gitignore` com o conteúdo do capítulo, e `.vscode/settings.json` se você usa VS Code. Depois, sem `--select` — as regras agora vêm do `pyproject.toml`:

```
ruff check bagunca.py
ruff format --check bagunca.py
```

```
All checks passed!
1 file already formatted
```

E o programa ficou melhor por decisão sua, não da ferramenta.

8. Experimente o depurador. Acrescente `breakpoint()` como primeira linha de `media` e rode `python bagunca.py`. Você cai no `pdb`, que mostra o arquivo, a linha e o comando onde parou:

```
> /home/voce/lab05/bagunca.py(2)media()
-> breakpoint()
(Pdb)
```

Digite `valores` para ver o argumento, `n` para avançar uma linha, `c` para continuar até o fim e `q` para sair:

```
(Pdb) valores
[1, 2, 3]
(Pdb) n
> /home/voce/lab05/bagunca.py(3)media()
-> return sum(valores) / len(valores)
(Pdb) c
2.0
```

O caminho na sua máquina será outro. Depois remova o `breakpoint()` — ou teste `PYTHONBREAKPOINT=0 python bagunca.py`, que ignora todos eles sem editar o arquivo.

9. O notebook e o mesmo interpretador.

```
python -m ipykernel install --user --name lab05 --display-name "Python (lab05)"
jupyter lab
```

No navegador, crie um notebook, escolha o kernel `Python (lab05)` e execute numa célula:

5. Ambiente de trabalho: editor, VS Code, Jupyter e o terminal

```
import sys
sys.executable
```

O resultado tem de ser o mesmo caminho do passo 1. Se for `/usr/bin/python3`, o kernel está errado — troque em **Kernel** → **Change Kernel**.

10. Reproduza a armadilha da ordem. Crie três células com o conteúdo A, B e C do capítulo. Execute na ordem **A, C, B, C** clicando célula por célula. Confirme que:

- a célula C falhou com **NameError** na primeira vez e funcionou na segunda;
- lendo a tela de cima para baixo, os contadores são [1], [3], [4] — a célula do meio rodou por último entre as duas primeiras, e a de baixo guarda apenas a execução mais recente. O [2], que foi a falha, **desapareceu da tela**: rodar C de novo sobrescreveu o resultado anterior.

Esse último detalhe é o que torna a armadilha perigosa. O erro aconteceu, você corrigiu por acidente, e o notebook não guarda memória de que houve problema.

11. Prove que está reproduzível. Use **Kernel** → **Restart Kernel and Run All Cells**. Agora os contadores são 1, 2, 3 de cima para baixo e nada falha, porque a ordem do arquivo passou a ser a ordem de execução. Faça isso sempre antes de mostrar um notebook para alguém.

12. Quebre de propósito para fixar. Numa célula nova, escreva `taxa = 0.25` e execute. Depois **apague a célula**. Rode a célula C: ela usa 0.25, um valor que não existe em nenhum lugar do arquivo. Reinicie o kernel e rode tudo: volta para 0.10. Esse é o fantasma que faz notebook não reproduzir.

13. Feche com deactivate (e Ctrl+C duas vezes no terminal do Jupyter).

5.9. Resumo

- Editor, terminal e kernel de notebook são três processos. Quando algo funciona num e não no outro, a pergunta é qual interpretador cada um usa: `import sys; print(sys.executable)` responde em dez segundos.
- Um ambiente virtual por projeto, e as três ferramentas apontando para ele. No VS Code isso se registra em `.vscode/settings.json`, que vai para o Git.
- Formatador arruma aparência; linter procura problema. O **ruff** faz as duas coisas e substitui a pilha antiga de ferramentas. A configuração vive no `pyproject.toml`.
- A automação tem um limite honesto: **ruff** remove import inútil e conserta espaçamento, mas não troca um laço por índice por `sum`, nem renomeia função. Arquivo aprovado pelo linter pode ser programa ruim.
- `breakpoint()` está na linguagem desde o 3.7 e substitui a depuração por `print`. `PYTHONBREAKPOINT=0` desliga todos.
- Notebook é para explorar. O contador `In [n]` conta execuções, não células: se não estiver em sequência, o notebook não é reproduzível. **Restart Kernel and Run All Cells** antes de dar por pronto; código com valor duradouro migra para `.py`.
- Versione a configuração (`pyproject.toml`, `.vscode/`, `.gitignore`), nunca o ambiente (`.venv/`, `__pycache__/`, checkpoints).

6. Sintaxe essencial: indentação, linhas lógicas e comentários

Você copiou um trecho de um site, colou no seu arquivo e o programa parou de rodar. A mensagem fala de indentação numa linha que parece idêntica às outras. Você apaga os espaços e digita de novo, um por um, e aí funciona — sem que nada tenha mudado na tela.

Mudou. O que estava lá era um caractere de tabulação e o que você digitou foram espaços; os dois desenham a mesma coisa e são bytes diferentes. Em quase toda linguagem isso seria irrelevante, porque o espaço em branco não significa nada. Em Python, o espaço em branco **é** a estrutura do programa, e este capítulo explica exatamente até onde essa afirmação vai.

6.1. O bloco é a indentação

Em C, Java, JavaScript e Rust, um bloco é o que está entre chaves; a indentação é enfeite para humanos. Em Python não existem chaves — e o `from __future__ import braces` do Capítulo 1 responde **not a chance** a quem pede. O bloco é definido pela quantidade de espaço à esquerda.

Isso não é convenção: é gramática, resolvida na análise léxica do Capítulo 2. O módulo `tokenize` mostra o mecanismo. Para este arquivo de três linhas:

```
if x:
    y = 1
z = 2
```

Os tokens que o Python produz são:

```
import tokenize

with open("t1.py", "rb") as f:
    for tok in tokenize.tokenize(f.readline):
        print(f"{tokenize.tok_name[tok.type]:<12} {tok.string!r}")
```

```
ENCODING      'utf-8'
NAME          'if'
NAME          'x'
OP            ':'
NEWLINE       '\n'
INDENT        '    '
NAME          'y'
OP            '='
NUMBER        '1'
```

6. Sintaxe essencial: indentação, linhas lógicas e comentários

```
NEWLINE      '\n'
DEDENT       ''
NAME         'z'
OP           '='
NUMBER       '2'
NEWLINE      '\n'
ENDMARKER    ''
```

Ali estão os tokens `INDENT` e `DEDENT`. Eles são o equivalente exato da chave que abre e da chave que fecha em outras linguagens — só que o Python os gera a partir do espaço em branco em vez de exigir que você os digite. O `DEDENT` sai com string vazia porque não corresponde a nenhum caractere: é uma marca posicional.

Figura 6.1 põe as duas notações lado a lado.

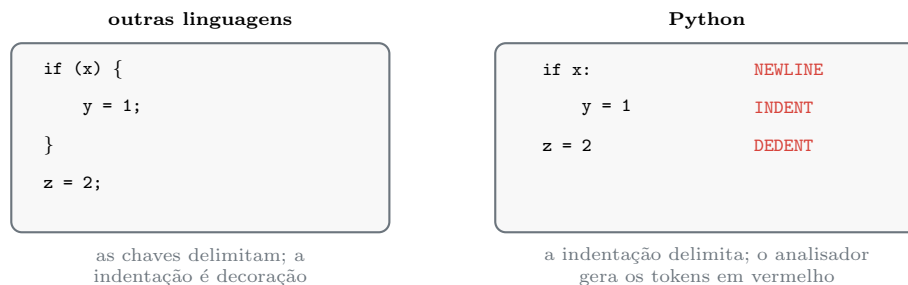


Figura 6.1.: `INDENT` e `DEDENT` são as chaves que você não digita.

A consequência é que **errar o espaço em branco é errar a sintaxe**, com mensagem de erro e tudo. Vale conhecer as quatro que aparecem.

Indentação onde não cabia bloco nenhum:

```
preco = 10  
    total = preco * 2
```

```
File "/tmp/lab06/e1.py", line 2  
    total = preco * 2  
IndentationError: unexpected indent
```

Bloco prometido e não entregue. Todo `:` no fim de uma linha abre um bloco, e o bloco tem de existir:

```
if preco > 5:  
print("caro")
```

```
File "/tmp/lab06/e2.py", line 2  
    print("caro")  
~~~~~  
IndentationError: expected an indented block after 'if' statement on line 1
```

Repare na precisão da mensagem: ela diz qual comando abriu o bloco e em que linha. Esse nível de detalhe é recente e economiza tempo.

Volta para um nível que não existe:

```
def f():
    a = 1
    b = 2
```

```
File "/tmp/lab06/e5.py", line 3
    b = 2
    ^
IndentationError: unindent does not match any outer indentation level
```

O `b` está a quatro espaços, mas o único nível aberto era o de oito. Não há para onde fechar.

Tabulação misturada com espaço — o erro da abertura do capítulo:

```
File "/tmp/lab06/e4.py", line 3
    b = 2
    ^
TabError: inconsistent use of tabs and spaces in indentation
```

`TabError` é uma subclasse de `IndentationError`, que por sua vez é subclasse de `SyntaxError`. Ou seja: os três são detectados na compilação, antes de qualquer linha rodar, exatamente como descrito no Capítulo 2.

Quanto ao tamanho da indentação, a linguagem só exige **consistência dentro do bloco**. Dois espaços funcionam:

```
def f():
    return 1

print(f())
```

```
1
```

Funciona e ninguém escreve assim. A PEP 8 pede **quatro espaços, nunca tabulação**, e essa é uma das poucas regras de estilo que o ecossistema trata como obrigatória: todo editor, todo formatador e todo projeto usam quatro espaços. Configure o editor para inserir espaços ao apertar Tab e o problema desaparece de vez.

6.2. Linha física e linha lógica

A segunda ideia estrutural do capítulo é que **uma instrução não precisa caber numa linha do arquivo**. O Python distingue a *linha física* (o que termina com Enter) da *linha lógica* (uma instrução completa).

Normalmente as duas coincidem. Dentro de parênteses, colchetes ou chaves, não:

6. Sintaxe essencial: indentação, linhas lógicas e comentários

```
clientes = [  
    "sitio-mangueira",  
    "fazenda-boa-vista",  
]  
  
total = (  
    10  
    + 20  
    + 30  
)
```

São duas linhas lógicas escritas em nove linhas físicas. Isso se chama **continuação implícita**, e é o mecanismo que os tokens revelam. Para o `total` acima:

ENCODING	'utf-8'
NAME	'total'
OP	'='
OP	'('
NL	'\n'
NUMBER	'1'
NL	'\n'
OP	'+'
NUMBER	'2'
NL	'\n'
OP	')'
NEWLINE	'\n'
ENDMARKER	''

Aqui está a distinção inteira, num lugar onde ela é visível: as quebras **dentro** dos parênteses viraram NL, e só a de depois do parêntese que fecha virou NEWLINE. NEWLINE termina uma linha lógica; NL é uma quebra de linha física que não termina nada. Uma linha lógica, quatro linhas físicas.

Figura 6.2 mostra o efeito.

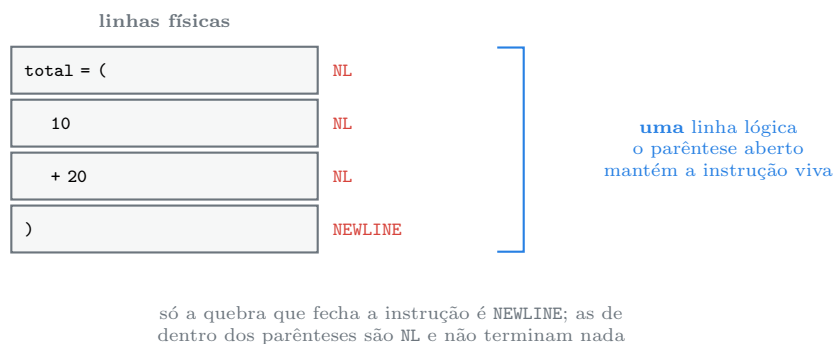


Figura 6.2.: Dentro de parênteses, a quebra de linha não termina a instrução.

Existe também a **continuação explícita**, com barra invertida no fim da linha:

```
soma = 1 + \
      2
```

Ela funciona, e a comunidade evita. O motivo é prático: um único espaço depois da barra invertida quebra o arquivo, e o espaço é invisível. Quando você precisa dividir uma linha, o caminho é acrescentar parênteses e usar a continuação implícita, que é robusta.

Isso resolve o problema real de manter linhas curtas. A PEP 8 pede no máximo 79 caracteres (as ferramentas modernas usam 88, como vimos no Capítulo 5), e a continuação implícita é como se obedece a esse limite sem ficar com código ilegível:

```
# em vez de uma linha de 120 caracteres
mensagem = f"cliente {cliente} teve {quedas} quedas entre {inicio} e {fim}"

# quebre dentro dos parênteses que já existem
mensagem = (
    f"cliente {cliente} teve {quedas} quedas "
    f"entre {inicio} e {fim}"
)
```

Duas strings literais adjacentes são concatenadas pelo compilador — outro caso de trabalho feito antes de o programa rodar, como o dobramento de constantes do Capítulo 2.

E o inverso, colocar várias instruções numa linha com ponto e vírgula, também é legal:

```
python3 -c "a = 1; b = 2; print(a + b)"
```

3

Legal e desaprovado. Em `python3 -c`, onde não há arquivo, é a única saída e está tudo bem. Em arquivo, é uma linha para cada instrução.

6.3. Comando e expressão

Uma distinção que evita confusão desde já: **expressão** é algo que tem valor; **comando** é algo que acontece. `2 + 3` é expressão. `x = 5` é comando. É por isso que o REPL do Capítulo 4 responde ao primeiro e fica calado com o segundo.

Onde a linguagem espera uma expressão, um comando é erro de sintaxe:

```
python3 -c "x = (z = 5)"
```

```

x = (z = 5)
~~~~~

```

```
SyntaxError: invalid syntax. Maybe you meant '==' or ':=' instead of '='?
```

A mensagem já sugere as duas coisas que provavelmente você queria. `==` compara e é expressão. E `:=`, o operador morsa do Capítulo 1, atribui e produz valor, o que o torna uma expressão:

```
python3 -c "x = (y := 5); print(x, y)"
```

```
5 5
```

Esse é justamente o motivo de o morsa ter sido criado: permitir atribuição em lugares que exigem expressão, como a condição de um `if` ou de um `while`. O Volume 3 volta ao assunto.

6.4. Nomes

Um identificador em Python começa com letra ou underscore e continua com letras, dígitos ou underscore. Não pode começar com dígito:

```
python3 -c "2gatos = 1"
```

```
  2gatos = 1
  ^
SyntaxError: invalid decimal literal
```

O que surpreende quem vem de outras linguagens é que “letra” quer dizer letra Unicode, não apenas ASCII. Isto é Python válido:

```
café = 3
preço_final = café * 2
print(preço_final)
```

```
6
```

Funciona, e este manual **não** recomenda. Nome de identificador em inglês ou em português sem acento, sempre — porque acento em nome de variável cria problemas de codificação em ferramentas antigas, dificulta digitar em teclado alheio e, pior, permite dois nomes visualmente idênticos e diferentes para o interpretador. Texto de mensagem, comentário e documentação, esses sim vão em português correto, com acento.

As palavras reservadas não podem ser usadas como nome. São 35:

```
import keyword

print(len(keyword.kwlist))
print(keyword.kwlist)
```

```
35
```

```
['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await', 'break',
↪  'class', 'continue', 'def', 'del', 'elif', 'else', 'except', 'finally',
↪  'for', 'from', 'global', 'if', 'import', 'in', 'is', 'lambda', 'nonlocal',
↪  'not', 'or', 'pass', 'raise', 'return', 'try', 'while', 'with', 'yield']
```


E existe uma segunda lista, curiosa e recente — as **palavras reservadas suaves**:

```
print(keyword.softkwlist)
```

```
['_', 'case', 'match', 'type']
```

Essas quatro só são especiais no contexto em que a gramática as espera. Fora dele, continuam sendo nomes comuns: `match = re.match(...)` é código perfeitamente válido, e é exatamente por isso que a lista suave existe. Quando o `match` foi acrescentado no Python 3.10, torná-lo palavra reservada de verdade quebraria uma quantidade enorme de código que já usava `match` como variável.

Sobre convenção de nomes, a PEP 8 é curta e o ecossistema segue à risca:

Tipo	Convenção	Exemplo
função, variável, módulo	snake_case	media_niveis, taxa_juros
classe	CamelCase	ConfiguracaoInvalida
constante	MAIUSCULA_COM_UNDERSCORE	TAXA_PADRAO
interno ao módulo	um underscore à frente	_cache

O **ruff** do Capítulo 5 verifica isso com as regras da família N, e foi o N802 que reclamou de `Media`.

6.5. Comentários e documentação

Comentário começa com o sinal de cerquilha e vale até o fim da linha física. Não existe comentário de bloco em Python.

O que existe, e é diferente, é a **docstring**: um literal de texto como primeira instrução de um módulo, função, classe ou método. A diferença não é estilística — comentário é descartado pelo compilador, docstring é um objeto guardado no atributo `__doc__`:

```
"""Documentacao do modulo."""
# comentario comum
```

```
def f():
    """Doc da funcao."""
    pass
```

```
python3 -c "import d; print(repr(d.__doc__)); print(repr(d.f.__doc__))"
```

```
'Documentacao do modulo.'
'Doc da funcao.'
```

6. Sintaxe essencial: indentação, linhas lógicas e comentários

O comentário não aparece em lugar nenhum; as duas docstrings estão acessíveis em tempo de execução. É delas que o **help** do Capítulo 4 tira o texto, e é nelas que geradores de documentação se apoiam. Um texto entre três aspas no meio de uma função, por outro lado, não é docstring: é uma expressão de string cujo valor é jogado fora — funciona como comentário multilinha por acidente, e não é para isso que serve.

Sobre o conteúdo dos comentários, a regra que economiza mais tempo de leitura: **comente o porquê, não o quê**.

```
# antipadrão: repete o código em português
# soma os níveis e divide pelo total
media = sum(níveis) / len(níveis)

# pythônico: explica o que o código não pode dizer
# o equipamento reporta -99 quando a porta esta apagada;
# incluir esses valores derrubaria a media do enlace
níveis = [n for n in níveis if n != -99]
```

O primeiro comentário vira mentira na primeira vez que alguém mudar a linha. O segundo carrega informação que não está em lugar nenhum do código.

E aquele **pass** do exemplo: ele existe porque a gramática exige um bloco depois do **:**, e às vezes você não tem nada para colocar ali ainda. **pass** é o comando que não faz nada, e serve exatamente para satisfazer a gramática.

6.6. Uma nota sobre codificação

Arquivo **.py** é **UTF-8 por padrão** desde o Python 3, e é por isso que **café** e os acentos deste manual funcionam sem cerimônia. Você ainda vai encontrar, em código antigo, uma linha como esta no topo:

```
# -*- coding: utf-8 -*-
```

É a declaração de codificação da PEP 263, obrigatória no Python 2, onde o padrão era ASCII. Hoje ela é redundante e pode ser apagada. Só faz diferença se o arquivo estiver **em outra** codificação, o que é um problema a resolver e não a documentar.

6.7. Jeito Pythônico

A PEP 8 é o guia de estilo da linguagem, e a maior parte dela se resume a coisas que uma ferramenta aplica sozinha. O resumo do que importa neste capítulo:

```
# pythônico
def media_níveis(níveis: list[float]) -> float:
    """Media dos níveis validos, ignorando portas apagadas."""
    validos = [n for n in níveis if n != -99]
    if not validos:
        raise ValueError("nenhum nível valido")
    return sum(validos) / len(validos)
```

Quatro espaços por nível, **snake_case**, docstring em vez de comentário no topo, duas linhas em branco entre funções de nível superior, uma instrução por linha, e nada de barra invertida.

Os antipadrões correspondentes:

```
# antipadrão: tabulação (ou mistura com espaço)
# antipadrão: continuação com barra invertida
mensagem = "cliente " + cliente + \
    " teve problemas"

# antipadrão: várias instruções numa linha
if not validos: raise ValueError("nenhum nível valido")

# antipadrão: comentário de bloco com três aspas no meio da função
"""
esta parte calcula a media
"""

# antipadrão: acento em identificador
média = sum(níveis) / len(níveis)
```

Sobre versões, o que mudou na sintaxe e a partir de quando:

- **Python 3.8** — o operador morsa `:=` (PEP 572), que faz da atribuição uma expressão.
- **Python 3.10** — `match` e `case` como palavras reservadas **suaves** (PEP 634), e mensagens de erro de sintaxe muito melhores, com o comando que abriu o bloco identificado por nome e linha.
- **Python 3.11** — marcação de coluna no traceback, com `~` e `^` apontando o trecho exato (PEP 657).
- **Python 3.12** — `type` entrou na lista de palavras suaves, junto com a sintaxe nova de genéricos (PEP 695), e f-strings deixaram de ter restrição sobre aspas e barras invertidas no interior (PEP 701).

Uma reflexão para fechar. A indentação obrigatória é a decisão de projeto mais criticada do Python por quem chega de fora, e a que a comunidade defende com mais convicção depois de um tempo. O argumento é simples: em toda linguagem, programador competente indenta o código de qualquer maneira. A diferença é que nas outras a indentação pode **mentir** — o bloco pode ser diferente do que os olhos veem, porque quem manda são as chaves. Em Python, o que você vê é o que executa. É o “legibilidade conta” da PEP 20 aplicado ao nível do caractere.

6.8. Laboratório

Objetivo: provocar cada erro de indentação de propósito, observar os tokens que o analisador produz e distinguir na prática linha física de linha lógica.

1. Ambiente.

```
mkdir -p ~/lab06 && cd ~/lab06
python3 -m venv .venv && source .venv/bin/activate
```

6. Sintaxe essencial: indentação, linhas lógicas e comentários

2. Provoque os quatro erros. Crie um arquivo para cada caso, rode e leia a mensagem inteira antes de corrigir. Este é o passo em que se aprende a reconhecer o erro pelo texto.

e1.py — indentação inesperada:

```
preco = 10
    total = preco * 2
```

e2.py — bloco que faltou:

```
preco = 10
if preco > 5:
    print("caro")
```

e5.py — volta para nível inexistente:

```
def f():
    a = 1
    b = 2
```

Confirme que os três dão `IndentationError` e que o texto de cada um é diferente.

3. Produza um `TabError`. Este exige cuidado, porque o editor pode converter tabulação em espaço automaticamente. Gere o arquivo pelo terminal, onde você controla os bytes:

```
printf 'def f():\n    a = 1\n\tb = 2\n' > e4.py
cat -A e4.py
python e4.py
```

O `cat -A` mostra os caracteres invisíveis: `^I` é tabulação e `$` é fim de linha. Você deve ver a terceira linha começando com `^I` e as outras com espaços. E o erro:

```
TabError: inconsistent use of tabs and spaces in indentation
```

4. Confirme o parentesco das exceções.

```
python -c "print(TabError.__mro__)"
```

```
(<class 'TabError'>, <class 'IndentationError'>, <class 'SyntaxError'>, <class
↳ 'Exception'>, <class 'BaseException'>, <class 'object'>)
```

`TabError` é um `IndentationError`, que é um `SyntaxError`. Por isso os três param o programa antes de a primeira linha rodar. A hierarquia de exceções é assunto do Capítulo 7 e do Volume 6.

5. Veja os tokens de indentação. Crie `t1.py` com o `if/y/z` do capítulo e rode o trecho do `tokenize`. Localize na saída o `INDENT` com quatro espaços e o `DEDENT` com string vazia.

6. Veja `NL` contra `NEWLINE`. Crie `t2.py`:

```
a = 1

# comentario
b = 2
```

Rode o mesmo `tokenize`. Você deve ver `NEWLINE` depois de cada instrução e `NL` na linha em branco e depois do comentário — porque nenhuma das duas termina instrução alguma. Agora `t3.py`:

```
total = (
    1
    + 2
)
```

Aqui há **um** `NEWLINE` só, no fim, e `NL` nas quebras internas. Uma linha lógica em quatro físicas.

7. Compare as duas continuações. Crie `cont.py` com o conteúdo do capítulo (a lista, o `total` entre parênteses e a soma com barra invertida) e rode:

```
python cont.py
```

```
['sitio-mangueira', 'fazenda-boa-vista'] 60 3
```

Agora **acrescente um espaço depois da barra invertida** da soma e rode de novo. O programa quebra, e a mensagem não ajuda muito, porque o caractere culpado é invisível. Desfaça. Foi por isso que a comunidade abandonou a barra invertida.

8. Explore as palavras reservadas.

```
python -c "import keyword; print(len(keyword.kwlist))"
python -c "import keyword; print(keyword.softkwlist)"
python -c "import keyword; print(keyword.iskeyword('match'),
↪ keyword.issoftkeyword('match'))"
```

A última linha deve responder `False True`: `match` não é palavra reservada, é suave. Confirme que dá para usá-la como variável:

```
python -c "match = 42; print(match)"
```

9. Teste um nome inválido e um exótico.

```
python -c "2gatos = 1"
python -c "café = 3; print(café * 2)"
```

O primeiro falha, o segundo imprime 6. Sabendo que funciona, continue não usando.

10. Prove que docstring é dado e comentário não. Crie `d.py` com o conteúdo do capítulo e rode:

```
python -c "import d; print(repr(d.__doc__)); print(repr(d.f.__doc__))"
python -c "import d; help(d.f)"
```

Não há forma de recuperar o comentário — ele não existe depois da compilação.

11. Deixe a ferramenta arrumar o que é mecânico.

```
python -m pip install ruff
printf 'x=1\ny  =  2\nif x>0:\n          print( x+y )\n' > torto.py
ruff format torto.py
cat torto.py
```

```
1 file reformatted
x = 1
y = 2
if x > 0:
    print(x + y)
```

O formatador normalizou os espaços e trouxe a indentação de oito para quatro. O que ele **não** faz é adivinhar a estrutura de um bloco errado:

```
printf 'x=1\nif x>0:\nprint(x)\n' > quebrado.py
ruff format quebrado.py; echo "exit=$?"
```

```
error: Failed to parse quebrado.py:3:1: Expected an indented block after `if`
↳ statement
exit=2
```

Arquivo que não compila não pode ser formatado — o formatador precisa da árvore sintática do Capítulo 2, e não há árvore aqui.

12. Feche com deactivate.

6.9. Resumo

- Em Python o bloco é a indentação, e isso é gramática: o analisador emite os tokens `INDENT` e `DEDENT`, que fazem o papel das chaves de outras linguagens. `tokenize` mostra isso.
- Quatro erros de espaço em branco valem reconhecer de cara: `unexpected indent`, `expected an indented block`, `unindent does not match any outer indentation level` e `TabError`. Todos são `SyntaxError` por herança e param o programa antes da primeira linha.
- A linguagem só exige consistência; a PEP 8 exige quatro espaços e nenhuma tabulação, e o ecossistema inteiro obedece.
- Linha física é o que termina com `Enter`; linha lógica é a instrução completa. Dentro de parênteses, colchetes ou chaves, a quebra vira `NL` e não termina nada — é a continuação implícita, e é como se quebra linha longa. Barra invertida funciona e se evita.
- Expressão tem valor, comando acontece. `=` é comando e não cabe onde se espera expressão; `==` compara e `:=` atribui produzindo valor.

- São 35 palavras reservadas e 4 suaves (`_`, `case`, `match`, `type`), que só valem no contexto da gramática e continuam servindo como nome comum.
- Comentário é descartado na compilação; docstring vira `__doc__` e alimenta o `help`. Comente o porquê, não o quê. Arquivo `.py` é UTF-8 por padrão, e `# -*- coding: utf-8 -*-` é herança do Python 2.

7. Erros e tracebacks: como ler o que o Python está dizendo

Um script que rodou a semana toda quebrou hoje de manhã. A tela cospe onze linhas de texto com caminhos de arquivo, setas e nomes que você não reconhece. A reação natural é rolar para o topo, procurar a primeira linha e tentar entender o que ela quer.

É o inverso do que funciona. Um traceback é organizado do mais antigo para o mais recente, e a informação que responde “o que aconteceu” está na **última** linha. Ler traceback é a habilidade que mais economiza tempo em Python, e a que menos se ensina.

7.1. O traceback do começo ao fim

Vamos quebrar um programa de verdade. `relatorio.py` calcula a média dos níveis de sinal óptico de um arquivo de log:

```
"""Gera o relatorio de quedas por cliente."""

from pathlib import Path

def ler_linhas(caminho):
    return Path(caminho).read_text(encoding="utf-8").splitlines()

def extrair_nivel(linha):
    campos = linha.split()
    return float(campos[3])

def media_niveis(caminho):
    linhas = ler_linhas(caminho)
    niveis = [extrair_nivel(l) for l in linhas]
    return sum(niveis) / len(niveis)

if __name__ == "__main__":
    print(media_niveis("sinais.log"))
```

E `sinais.log` tem um campo estragado na terceira linha — o equipamento não conseguiu ler e escreveu texto no lugar do número:

7. Erros e tracebacks: como ler o que o Python está dizendo

```
2026-03-11 03:14:02 sitio-mangueira -18.4
2026-03-11 05:02:19 fazenda-boavista -21.7
2026-03-11 06:47:55 chacara-do-ze sem-leitura
```

```
python3 relatorio.py
```

```
Traceback (most recent call last):
  File "/tmp/lab07/relatorio.py", line 22, in <module>
    print(media_niveis("sinais.log"))
    ~~~~~~^~~~~~
  File "/tmp/lab07/relatorio.py", line 17, in media_niveis
    niveis = [extrair_nivel(l) for l in linhas]
    ~~~~~~^~
  File "/tmp/lab07/relatorio.py", line 12, in extrair_nivel
    return float(campos[3])
ValueError: could not convert string to float: 'sem-leitura'
```

Esse texto tem uma estrutura fixa, e vale nomear cada parte.

A primeira linha é um aviso de ordenação, não informação sobre o erro: `most recent call last` significa “a chamada mais recente está por último”. É o Python dizendo onde olhar, e quase todo mundo ignora.

No meio vêm os quadros (*frames*), um por chamada de função que estava em andamento. Cada quadro tem três linhas: o arquivo e a linha, o nome da função (`in media_niveis`), o código daquela linha, e — desde o Python 3.11 — a marcação de coluna com til e circunflexo apontando o trecho exato da expressão que estava sendo avaliado.

A última linha é a resposta: o tipo da exceção (`ValueError`), dois pontos, e a mensagem. `could not convert string to float: 'sem-leitura'` diz o que deu errado e qual dado causou o problema.

Figura 7.1 organiza a leitura.

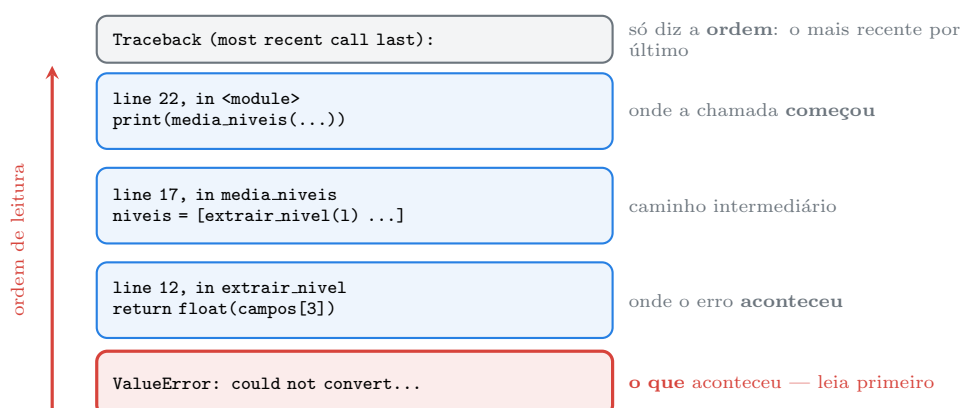


Figura 7.1.: O traceback imprime do mais antigo ao mais recente; leia de baixo para cima.

A ordem prática de leitura é esta:

1. **A última linha**, sempre. Tipo e mensagem respondem o que aconteceu.

2. **O quadro de baixo**, que é onde o erro estourou.
3. **Subindo**, até encontrar o primeiro quadro que seja **código seu**. É ali que você tem poder de agir.

O passo 3 importa mais do que parece, porque tracebacks reais atravessam bibliotecas. Se o arquivo de log não existir:

```
python3 -c "from relatorio import media_niveis; media_niveis('naoexiste.log')"
```

```
Traceback (most recent call last):
  File "<string>", line 1, in <module>
    media_niveis('naoexiste.log')
    ~~~~~~^~~~~~
  File "/tmp/lab07/relatorio.py", line 16, in media_niveis
    linhas = ler_linhas(caminho)
  File "/tmp/lab07/relatorio.py", line 7, in ler_linhas
    return Path(caminho).read_text(encoding="utf-8").splitlines()
           ~~~~~~^~~~~~
  File "/usr/lib/python3.13/pathlib/_local.py", line 548, in read_text
    return PathBase.read_text(self, encoding, errors, newline)
           ~~~~~~^~~~~~
  File "/usr/lib/python3.13/pathlib/_abc.py", line 632, in read_text
    with self.open(mode='r', encoding=encoding, errors=errors, newline=newline)
↪   as f:
       ~~~~~~^~~~~~
  File "/usr/lib/python3.13/pathlib/_local.py", line 539, in open
    return io.open(self, mode, buffering, encoding, errors, newline)
           ~~~~~~^~~~~~
FileNotFoundError: [Errno 2] No such file or directory: 'naoexiste.log'
```

São sete quadros, e **quatro deles são da biblioteca padrão**. Nada em `/usr/lib/python3.13/pathlib/` está com defeito — aqueles quadros são só o caminho que a chamada percorreu. O quadro que interessa é o `ler_linhas`, na linha 7 do seu arquivo, e a informação decisiva continua sendo a última linha.

Daí a heurística que resolve a maioria dos casos: **procure o quadro mais baixo cujo caminho seja do seu projeto**. Erro dentro de biblioteca madura quase sempre significa que você passou para ela um valor que ela não esperava.

7.2. As exceções que você vai ver mais

Cada tipo de exceção é uma hipótese sobre a causa. Reconhecê-las de imediato é metade do trabalho.

NameError — o nome não existe. Desde o Python 3.10, com sugestão:

```
comprimento = 10
print(compriemnto)
```

7. Erros e tracebacks: como ler o que o Python está dizendo

```
NameError: name 'compriemnto' is not defined. Did you mean: 'comprimento'?
```

AttributeError — o objeto não tem aquele atributo ou método. Também sugere, quando o nome é parecido o bastante:

```
AttributeError: 'str' object has no attribute 'upperr'. Did you mean: 'upper'?
AttributeError: 'str' object has no attribute 'startswith'. Did you mean:
↳ 'startswith'?
```

E não sugere, quando o palpite é distante demais:

```
AttributeError: 'str' object has no attribute 'uppercase'
```

A sugestão vem de uma medida de semelhança entre textos: `upperr` e `uppe` chegam perto de `upper`, mas `uppercase` não. Ausência de sugestão não significa que o método não exista com outro nome — significa apenas que a heurística não achou nada próximo. Aí é `dir` e `help`, como no Capítulo 4.

TypeError — a operação não faz sentido para aqueles tipos:

```
File "<string>", line 1, in <module>
    print('3' + 4)
          ~~~~^~~~
TypeError: can only concatenate str (not "int") to str
```

Aqui a marcação de coluna se mostra útil: o `^` está exatamente sobre o `+`, e os `~` cobrem os operandos. Esse é o “explícito é melhor que implícito” do Capítulo 1 em ação — a linguagem se recusa a adivinhar se você queria `"34"` ou `7`.

ValueError — o tipo está certo e o valor não serve. `float("sem-leitura")` recebeu um texto, que é o tipo esperado, com conteúdo impossível de converter. A dupla `TypeError` e `ValueError` confunde no começo, e a distinção é essa: tipo errado contra valor errado.

KeyError e **IndexError** — a chave ou a posição não existe:

```
print(d['quedas'])
      ~~~~~
KeyError: 'quedas'
```

```
print(campos[5])
      ~~~~~
IndexError: list index out of range
```

IndexError é o que você recebe quando o arquivo de log tem uma linha com menos campos do que o esperado — e é por isso que `campos[3]` em `extrair_nivel` é uma bomba armada, assunto do laboratório.

ZeroDivisionError — divisão por zero. No nosso programa, ele aparece se o arquivo estiver vazio:

```
print(sum(niveis)/len(niveis))
~~~~~^~~~~~
ZeroDivisionError: division by zero
```

FileNotFoundError — arquivo ou diretório inexistente, com o [Errno 2] que vem do sistema operacional.

ModuleNotFoundError — o import falhou:

```
import requestss
ModuleNotFoundError: No module named 'requestss'
```

Nove vezes em dez a causa não é o nome: é o ambiente virtual errado. Antes de investigar qualquer coisa, rode o `import sys; print(sys.executable)` do Capítulo 5.

7.3. Exceções encadeadas

Quando uma exceção acontece **enquanto** outra estava sendo tratada, o Python mostra as duas. É a parte do traceback que mais assusta e a mais informativa.

7.3.1. Encadeamento implícito

```
dados = {"cliente": "sitio"}

try:
    quedas = dados["quedas"]
except KeyError:
    print("nao achei a chave, vou tentar converter")
    quedas = int(dados["cliente"])
```

```
nao achei a chave, vou tentar converter
Traceback (most recent call last):
  File "/tmp/lab07/enc1.py", line 4, in <module>
    quedas = dados["quedas"]
    ~~~~~^~~~~~
KeyError: 'quedas'

During handling of the above exception, another exception occurred:

Traceback (most recent call last):
  File "/tmp/lab07/enc1.py", line 7, in <module>
    quedas = int(dados["cliente"])
ValueError: invalid literal for int() with base 10: 'sitio'
```

A frase-chave é “**During handling of the above exception**”: durante o tratamento da exceção acima, outra ocorreu. Isso quase sempre indica que o seu `except` tem um bug — o plano B falhou. O erro que você precisa corrigir é o de baixo; o de cima explica por que o programa chegou lá.

7.3.2. Encadeamento explícito

O caso oposto é intencional. Você captura um erro técnico e o traduz para um erro que fala a língua da sua aplicação, preservando o original com `raise ... from`:

```
class ConfiguracaoInvalida(Exception):
    """Erro de dominio da aplicacao."""

def ler_taxa(dados):
    try:
        return float(dados["taxa"])
    except KeyError as erro:
        raise ConfiguracaoInvalida("falta a chave 'taxa'") from erro

ler_taxa({"cliente": "sitio"})
```

```
Traceback (most recent call last):
  File "/tmp/lab07/enc2.py", line 7, in ler_taxa
    return float(dados["taxa"])
          ~~~~~~^~~~~~
KeyError: 'taxa'
```

The above exception was the direct cause of the following exception:

```
Traceback (most recent call last):
  File "/tmp/lab07/enc2.py", line 12, in <module>
    ler_taxa({"cliente": "sitio"})
    ~~~~~~^~~~~~
  File "/tmp/lab07/enc2.py", line 9, in ler_taxa
    raise ConfiguracaoInvalida("falta a chave 'taxa'") from erro
ConfiguracaoInvalida: falta a chave 'taxa'
```

Agora a frase é “**The above exception was the direct cause**” — causa direta, não acidente. Quem lê recebe as duas informações: a mensagem de alto nível (“falta a chave taxa”) e o detalhe técnico (“`KeyError: 'taxa'`”).

A terceira forma é suprimir a origem com `from None`, quando o erro interno é ruído:

```
def ler_taxa(dados):
    try:
        return float(dados["taxa"])
    except KeyError:
        raise ValueError("falta a chave 'taxa'") from None
```

```
Traceback (most recent call last):
  File "/tmp/lab07/enc3.py", line 8, in <module>
    ler_taxa({})
    ~~~~~~^~~~~~
```

```
File "/tmp/lab07/enc3.py", line 5, in ler_taxa
    raise ValueError("falta a chave 'taxa'") from None
ValueError: falta a chave 'taxa'
```

Limpo — e você jogou fora informação. Use com parcimônia. Figura 7.2 resume as três.



Figura 7.2.: Três encadeamentos: acidente, tradução proposital e supressão.

7.4. Quando o traceback é gigante

Recursão infinita produz o traceback mais intimidador que existe — e o CPython tem a gentileza de resumi-lo:

```
def profundidade(n):
    return profundidade(n + 1)

profundidade(0)
```

```
Traceback (most recent call last):
  File "/tmp/lab07/rec.py", line 5, in <module>
    profundidade(0)
    ~~~~~^~~~
  File "/tmp/lab07/rec.py", line 2, in profundidade
    return profundidade(n + 1)
  File "/tmp/lab07/rec.py", line 2, in profundidade
    return profundidade(n + 1)
  [Previous line repeated 996 more times]
RecursionError: maximum recursion depth exceeded
```

Doze linhas, não mil. O [Previous line repeated 996 more times] substitui os quadros repetidos. `RecursionError` quase sempre significa uma de duas coisas: falta a condição de parada, ou existe um ciclo entre funções que chamam umas às outras.

7.5. SyntaxError é diferente de todos os outros

Vale reforçar a distinção do Capítulo 2 e do Capítulo 6, porque ela muda como se procura o problema:

7. Erros e tracebacks: como ler o que o Python está dizendo

```
File "/tmp/lab06/e2.py", line 2
    print("caro")
    ~~~~~
IndentationError: expected an indented block after 'if' statement on line 1
```

Não há `Traceback` (most recent call last) e não há quadros. Não pode haver: nenhuma função foi chamada, porque o programa nunca começou. `SyntaxError` e suas subclasses são detectadas na compilação; todo o resto acontece em execução. Se você vê quadros, o programa rodou; se não vê, nem chegou a rodar.

7.6. Jeito Pythônico

A postura pythônica diante de erro tem um nome: **EAFP**, de *Easier to Ask Forgiveness than Permission* — é mais fácil pedir perdão do que permissão. O oposto é o **LBYL**, *Look Before You Leap*, olhe antes de saltar, comum em outras linguagens.

```
# LBYL: verifica tudo antes
import os

if os.path.exists(caminho):
    if os.access(caminho, os.R_OK):
        with open(caminho) as f:
            dados = f.read()
    else:
        dados = ""
else:
    dados = ""

# EAFP: tenta, e trata o que der errado
try:
    dados = Path(caminho).read_text(encoding="utf-8")
except (FileNotFoundError, PermissionError):
    dados = ""
```

A segunda não é só mais curta: é **correta**, e a primeira não é. Entre o `os.path.exists` e o `open` existe um intervalo em que o arquivo pode ser apagado por outro processo — uma condição de corrida que a verificação prévia não elimina, só torna mais rara. A exceção é a única forma de saber que a operação falhou de fato.

Três antipadrões que valem nome:

```
# antipadrão: engolir tudo em silêncio
try:
    nivel = float(campos[3])
except:
    pass
```


Esse é o pior pedaço de código deste capítulo. O `except` sem tipo captura **tudo**, incluindo `KeyboardInterrupt` — o seu `Ctrl+C` — e `SystemExit`. Com `pass` no corpo, o erro desaparece sem rastro, e é o “erros nunca devem passar em silêncio” da PEP 20 sendo violado da forma mais direta possível. O programa continua com dados errados e você descobre semanas depois.

```
# antipadrão: capturar Exception por precaução
try:
    nivel = float(campos[3])
except Exception as erro:
    print("deu erro:", erro)
```

Melhor, porque não engole o `Ctrl+C` e ao menos avisa. Ainda ruim, porque trata igual coisas diferentes: campo com texto inválido, arquivo corrompido e um bug de digitação no nome de uma variável.

```
# pythônico: capture o que você sabe tratar, o mais específico possível
try:
    nivel = float(campos[3])
except (IndexError, ValueError) as erro:
    logging.warning("linha ignorada (%s): %r", erro, linha)
    continue
```

`IndexError` para a linha com poucos campos, `ValueError` para o campo não numérico, e um registro do que foi descartado. Qualquer outra exceção continua subindo — porque não é caso previsto, e programa que quebra alto é melhor que programa que mente.

Duas regras finais:

Nunca use `assert` para validar entrada. Como visto no Capítulo 2, `python -O` remove as asserções. Validação é `if` com `raise`.

Prefira exceção específica à mensagem genérica. Quando você levanta um erro, escolha o tipo mais preciso da hierarquia (`ValueError` para valor ruim, `TypeError` para tipo errado, `KeyError` para chave ausente) ou crie a sua, herdando de `Exception`. O Volume 6 trata da hierarquia completa e de como desenhar exceções próprias.

Sobre versões — a qualidade das mensagens de erro é uma das áreas em que o Python mais melhorou:

- **Python 3.10** — `NameError` e `AttributeError` passaram a sugerir nomes parecidos; mensagens de `SyntaxError` identificam o comando e a linha que abriram o bloco.
- **Python 3.11** — marcação de coluna com `~` e `^` (PEP 657) e `ExceptionGroup` com `except*` (PEP 654), para quando várias exceções acontecem em paralelo. `add_note` permite anexar contexto a uma exceção já levantada.
- **Python 3.12** — `sys.last_exc` guarda a última exceção no modo interativo.
- **Python 3.13** — tracebacks **coloridos** por padrão no terminal, com o tipo e a mensagem destacados. `PYTHON_COLORS=0` desliga, o que é necessário ao redirecionar para arquivo em ferramenta que não entende as sequências de cor.

7.7. Laboratório

Objetivo: provocar cada exceção comum de propósito, praticar a leitura de baixo para cima e transformar um programa frágil num programa que sobrevive a dados ruins.

1. Ambiente e programa.

```
mkdir -p ~/lab07 && cd ~/lab07
python3 -m venv .venv && source .venv/bin/activate
```

Crie `relatorio.py` e `sinais.log` com o conteúdo do capítulo.

2. Leia o traceback na ordem certa.

```
python relatorio.py
```

Antes de olhar o resto, responda três perguntas usando **só a última linha**: qual é o tipo do erro, o que ele diz e qual dado causou. Depois vá ao quadro de baixo e confirme em que função e em que linha aconteceu.

3. Conte os quadros de um erro que atravessa a biblioteca.

```
python -c "from relatorio import media_niveis; media_niveis('naoexiste.log')"
```

Conte quantos quadros são seus e quantos são de `/usr/lib`. Encontre o quadro mais baixo do seu arquivo — é ali que a correção entra.

4. Provoque o `IndexError` escondido. Uma linha com menos campos que o esperado é outra bomba armada em `campos[3]`. Para vê-la, use um arquivo **sem** a linha **sem-leitura**, senão o `ValueError` do passo anterior estoura primeiro e esconde este:

```
cat > curto.log <<'EOF'
2026-03-11 03:14:02 sitio-mangueira -18.4
2026-03-11 22:00:00 vila-sao-joao
EOF
python -c "from relatorio import media_niveis; media_niveis('curto.log')"
```

```
File "/tmp/lab07/relatorio.py", line 12, in extrair_nivel
    return float(campos[3])
           ~~~~~~^^~
IndexError: list index out of range
```

Duas coisas para notar. O traceback é quase idêntico ao do passo 2, com um tipo diferente na última linha — mais uma razão para lê-la primeiro. E a marcação de coluna mudou de lugar: agora aponta `campos[3]`, e não a chamada de `float`, porque o erro aconteceu na indexação, antes de `float` ser chamada.

O fato de o primeiro erro esconder o segundo é a razão de o passo 12 tratar os dois de uma vez.

5. Provoque o `ZeroDivisionError`.

```
: > vazio.log
python -c "from relatorio import media_niveis; media_niveis('vazio.log')"
```

O : > cria um arquivo vazio. Sem linhas, `len(niveis)` é zero.

6. Colecione as exceções comuns. Rode cada uma e leia a mensagem inteira:

```
python -c "comprimento = 10; print(compriemnto)"
python -c "'manual'.upperr()"
python -c "'manual'.uppercase()"
python -c "print('3' + 4)"
python -c "print({'cliente': 'sitio'}['quedas'])"
python -c "print(['a','b'][5])"
python -c "import requestss"
```

Compare a segunda com a terceira: uma sugere `upper`, a outra não. A semelhança entre os nomes é o critério.

7. Veja o encadeamento implícito. Crie `enc1.py` com o conteúdo do capítulo e rode. Localize a frase `During handling of the above exception`. Pergunte-se qual dos dois erros você corrigiria — e a resposta é o de baixo, porque o `except` é que está errado.

8. Veja o encadeamento explícito. Crie `enc2.py` e rode. Localize `The above exception was the direct cause`. Depois crie `enc3.py`, com `from None`, e compare: a segunda saída é mais limpa e esconde que a causa raiz era um `KeyError`.

9. Provoque o `RecursionError`.

```
python -c "
def profundidade(n):
    return profundidade(n + 1)

profundidade(0)
" 2>&1 | tail -4
```

Confirme o `[Previous line repeated ... more times]`. Depois conte o traceback inteiro:

```
python -c "
def profundidade(n):
    return profundidade(n + 1)

profundidade(0)
" 2>&1 | wc -l
```

Uma dúzia de linhas para mil chamadas.

10. Confirme que `SyntaxError` não tem quadros.

```
printf 'if True:\nprint(1)\n' > sint.py
python sint.py
```

7. Erros e tracebacks: como ler o que o Python está dizendo

Nenhum Traceback (most recent call last), nenhum quadro. O programa não rodou.

11. Entenda as cores. No 3.13 o traceback sai colorido — mas só quando a saída é um terminal de verdade. Confirme que redirecionar já desliga:

```
python relatorio.py 2> err.txt
cat -v err.txt | tail -2
```

```
    return float(campos[3])
ValueError: could not convert string to float: 'sem-leitura'
```

Texto puro, sem sequência de escape nenhuma: o Python detectou que não havia terminal e não coloriu. Para **ver** a cor que o terminal recebe, force um pseudoterminal com o utilitário `script`:

```
script -q -c "python relatorio.py" /dev/null 2>&1 | cat -v | tail -1
```

```
^[[1;35mValueError^[[0m: ^[[35mcould not convert string to float:
↪  'sem-leitura'^[[0m^M
```

Ali estão os códigos de cor em volta do tipo e da mensagem. E é assim que se desliga:

```
PYTHON_COLORS=0 script -q -c "python relatorio.py" /dev/null 2>&1 | cat -v |
↪  tail -1
```

```
ValueError: could not convert string to float: 'sem-leitura'^M
```

Como o desligamento automático já cobre redirecionamento, `PYTHON_COLORS=0` serve para o caso menos comum: uma ferramenta de CI que aloca um pseudoterminal e depois grava o resultado num log que ninguém colore de volta.

12. Conserte o programa. Reescreva `extrair_nivel` e `media_niveis` para sobreviver a dados ruins, aplicando o EAEP com exceções específicas:

```
import logging

def extrair_nivel(linha):
    campos = linha.split()
    try:
        return float(campos[3])
    except (IndexError, ValueError) as erro:
        logging.warning("linha ignorada (%s): %r", erro, linha)
        return None

def media_niveis(caminho):
    linhas = ler_linhas(caminho)
```

```
niveis = [n for l in linhas if (n := extrair_nivel(l)) is not None]
if not niveis:
    raise ValueError(f"nenhum nivel valido em {caminho}")
return sum(niveis) / len(niveis)
```

Salve como `relatorio2.py` (com `import logging` no topo) e junte os dois problemas num arquivo só:

```
echo "2026-03-11 22:00:00 vila-sao-joao" >> sinais.log
python relatorio2.py
```

```
WARNING:root:linha ignorada (could not convert string to float: 'sem-leitura'):
↳ '2026-03-11 06:47:55 chacara-do-ze sem-leitura'
WARNING:root:linha ignorada (list index out of range): '2026-03-11 22:00:00
↳ vila-sao-joao'
-20.049999999999997
```

As duas linhas ruins foram avisadas e descartadas — cada uma com o erro que a derrubou e o conteúdo exato da linha — e a média saiu das duas boas. Compare com o comportamento anterior, em que a primeira linha ruim matava o programa e a segunda nem chegava a ser vista.

E o arquivo vazio:

```
python -c "from relatorio2 import media_niveis; media_niveis('vazio.log')"
```

```
File "/tmp/lab07/relatorio2.py", line 24, in media_niveis
    raise ValueError(f"nenhum nivel valido em {caminho}")
ValueError: nenhum nivel valido em vazio.log
```

Agora o erro é um `ValueError` com mensagem sua, que diz **qual** arquivo, em vez de um `ZeroDivisionError` cru vindo de uma divisão que o leitor do log não fez ideia de onde veio.

E o `-20.049999999999997` no lugar de `-20.05`? É o ponto flutuante binário outra vez, como no notebook do Capítulo 5. O Volume 2 explica.

Repare no `:=` do Capítulo 6 dentro da compreensão: ele chama `extrair_nivel` uma vez, guarda o resultado e o testa. Sem ele, seria preciso chamar duas vezes ou escrever um laço.

13. Feche com `deactivate`.

7.8. Resumo

- Traceback imprime do mais antigo ao mais recente. Leia de baixo para cima: primeiro a última linha (tipo e mensagem), depois o quadro mais baixo, depois subindo até o primeiro quadro do seu projeto.
- Quadro em `/usr/lib` quase nunca é bug da biblioteca: é o caminho da chamada. O quadro que interessa é o mais baixo que seja código seu.

7. Erros e tracebacks: como ler o que o Python está dizendo

- Desde o 3.11, a marcação com `~` e `^` aponta a coluna exata da expressão. Desde o 3.10, `NameError` e `AttributeError` sugerem nomes parecidos — e a ausência de sugestão só quer dizer que nada ficou próximo o bastante.
- `TypeError` é tipo errado, `ValueError` é valor errado; `KeyError` e `IndexError` são chave e posição inexistentes; `ModuleNotFoundError` é quase sempre ambiente virtual errado.
- “During handling of the above exception” indica que o seu `except` falhou — corrija o erro de baixo. “The above exception was the direct cause” é `raise ... from`, tradução proposital. `from None` limpa a saída e joga informação fora.
- `SyntaxError` e subclasses não têm quadros, porque o programa nunca começou. Ver quadros significa que ele rodou.
- EAFP vence LBYL: tentar e tratar a exceção é mais curto e mais correto que verificar antes, que sofre de condição de corrida. Capture o tipo mais específico que você sabe tratar; `except:` pelado e `except Exception:` `pass` são os piores hábitos possíveis.

Parte II.

**Volume 2 — Tipos, Operadores e
Expressões**

8. Objetos, variáveis e o modelo de referências

(em elaboracao)

9. Números inteiros, de ponto flutuante e complexos

(em elaboracao)

10. Precisão numérica: decimal, fractions e as armadilhas do float

(em elaboracao)

11. Booleanos, valor-verdade e os operadores lógicos

(em elaboracao)

12. Strings: criação, indexação e fatiamento

(em elaboracao)

13. Métodos de string, formatação e f-strings

(em elaboracao)

14. Unicode, bytes e codificação de caracteres

(em elaboracao)

Parte III.

**Volume 3 — Controle de Fluxo e
Funções**

15. Condicionais: if, elif, else e a expressão condicional

(em elaboracao)

16. Laços: while, for e o protocolo de iteração

(em elaboracao)

17. break, continue, else em laços e o casamento de padrões

(em elaboracao)

18. Definindo funções: parâmetros, argumentos e retorno

(em elaboracao)

19. Argumentos nomeados, valores padrão, *args e **kwargs

(em elaboracao)

20. Escopo, closures e a regra LEGB

(em elaboracao)

21. Funções como objetos: lambda, map, filter e ordem superior

(em elaboracao)

Parte IV.

**Volume 4 — Estruturas de Dados
Nativas**

22. Listas: operações, mutabilidade e cópia

(em elaboracao)

23. Tuplas, empacotamento e desempacotamento

(em elaboracao)

24. Dicionários: chaves, ordem e os métodos essenciais

(em elaboracao)

25. Conjuntos e operações de conjunto

(em elaboracao)

26. Compreensões de lista, dicionário e conjunto

(em elaboracao)

27. Ordenação, chaves de ordenação e comparação de objetos

(em elaboracao)

28. Escolhendo a estrutura certa: custo das operações

(em elaboracao)

Parte V.

**Volume 5 — Módulos, Pacotes e o
Ambiente**

29. Módulos, import e o caminho de busca

(em elaboracao)

30. Pacotes, `__init__.py` e a organização de um projeto

(em elaboracao)

31. Ambientes virtuais: venv, pip e o isolamento de dependências

(em elaboracao)

32. Empacotamento moderno: pyproject.toml, build e publicação

(em elaboracao)

33. uv, pipx e Poetry: o ecossistema de gerenciamento

(em elaboracao)

34. Ponto de entrada: `__main__`, scripts de console e a linha de comando

(em elaboracao)

Parte VI.

**Volume 6 — Arquivos, Erros e
Contextos**

35. Leitura e escrita de arquivos de texto

(em elaboracao)

36. Arquivos binários, buffers e o módulo io

(em elaboracao)

37. Caminhos e o sistema de arquivos com pathlib

(em elaboracao)

38. Exceções: hierarquia e o bloco try, except, else e finally

(em elaboracao)

39. Levantando exceções e criando exceções próprias

(em elaboracao)

40. Gerenciadores de contexto e a instrução with

(em elaboracao)

Parte VII.

**Volume 7 — Programação Orientada a
Objetos**

41. Classes, instâncias e o self

(em elaboracao)

42. Atributos, propriedades e encapsulamento

(em elaboracao)

43. Herança, MRO e a função super

(em elaboracao)

44. Composição, delegação e quando não herdar

(em elaboracao)

45. Métodos estáticos, métodos de classe e o desenho da API

(em elaboracao)

46. Dataclasses, NamedTuple e enumerações

(em elaboracao)

47. Classes abstratas, protocolos e duck typing

(em elaboracao)

Parte VIII.

**Volume 8 — O Modelo de Dados de
Python**

48. Métodos especiais: o protocolo por trás da sintaxe

(em elaboracao)

49. Iteradores e o protocolo de iteração

(em elaboracao)

50. Geradores, yield e a avaliação preguiçosa

(em elaboracao)

51. Decoradores de função

(em elaboracao)

52. Decoradores de classe, functools e metadados preservados

(em elaboracao)

53. Descritores, `__slots__` e o acesso a atributos

(em elaboracao)

54. Metaclasses e `__init_subclass__`: quando (não) usar

(em elaboracao)

Parte IX.

**Volume 9 — A Biblioteca Padrão
Essencial**

55. os, sys e a interação com o sistema

(em elaboracao)

56. datetime, zoneinfo e o tratamento de tempo

(em elaboracao)

57. collections: deque, Counter, defaultdict e namedtuple

(em elaboracao)

58. itertools e functools: a caixa de ferramentas funcional

(em elaboracao)

59. math, statistics e random

(em elaboracao)

60. logging: registrando o que o programa faz

(em elaboracao)

Parte X.

**Volume 10 — Texto, Dados e
Serialização**

61. Expressões regulares com o módulo re

(em elaboracao)

62. JSON, CSV e formatos tabulares

(em elaboracao)

63. YAML, TOML e arquivos de configuração

(em elaboracao)

64. Serialização binária: pickle, struct e os riscos envolvidos

(em elaboracao)

65. SQLite e a DB-API: banco de dados sem servidor

(em elaboracao)

66. hashlib, secrets e o tratamento de dados sensíveis

(em elaboracao)

Parte XI.

**Volume 11 — Qualidade: Testes,
Tipagem e Ferramentas**

67. Estilo, PEP 8 e formatação automática com Ruff e Black

(em elaboracao)

68. Anotações de tipo: fundamentos e o módulo typing

(em elaboracao)

69. Tipagem avançada: genéricos, protocolos e mypy

(em elaboracao)

70. Testes com pytest: fundamentos, asserções e fixtures

(em elaboracao)

71. Parametrização, duplões de teste e cobertura

(em elaboracao)

72. Depuração: pdb, breakpoint e o traceback difícil

(em elaboracao)

73. Documentação: docstrings, doctest e Sphinx

(em elaboracao)

Parte XII.

**Volume 12 — Automação, Sistema e
Redes**

74. Scripts de automação: arquivos, planilhas e tarefas repetitivas

(em elaboracao)

75. subprocess: chamando programas externos com segurança

(em elaboracao)

76. Interfaces de linha de comando com argparse, Click e Typer

(em elaboracao)

77. Redes com sockets: TCP, UDP e o modelo cliente-servidor

(em elaboracao)

78. Requisições HTTP com requests e httpx

(em elaboracao)

79. Raspagem de dados responsável: HTML, seletores e limites éticos

(em elaboracao)

Parte XIII.

**Volume 13 — Dados, Análise e
Visualização**

80. NumPy: arrays, vetorização e broadcasting

(em elaboracao)

81. pandas: Series, DataFrame e a carga de dados

(em elaboracao)

82. Limpeza, transformação e agregação com pandas

(em elaboracao)

83. Visualização de dados com Matplotlib

(em elaboracao)

84. SciPy e a computação científica aplicada

(em elaboracao)

85. Introdução ao aprendizado de máquina com scikit-learn

(em elaboracao)

Parte XIV.

**Volume 14 — Web, APIs e Bancos de
Dados**

86. Como funciona uma aplicação web: HTTP, WSGI e ASGI

(em elaboracao)

87. Validação de dados com Pydantic

(em elaboracao)

88. APIs REST com FastAPI

(em elaboracao)

89. Flask e Django: quando usar cada um

(em elaboracao)

90. Bancos de dados relacionais com SQLAlchemy

(em elaboracao)

91. Autenticação, autorização e segurança em aplicações Python

(em elaboracao)

92. Implantação: contêineres, servidores de aplicação e produção

(em elaboracao)

Parte XV.

**Volume 15 — Concorrência,
Desempenho e Internos**

93. Concorrência, paralelismo e o GIL

(em elaboracao)

94. Threads e o módulo threading

(em elaboracao)

95. Multiprocessing e o paralelismo real

(em elaboracao)

96. Programação assíncrona: async, await e asyncio

(em elaboracao)

97. Medindo desempenho: timeit, profiling e otimização

(em elaboracao)

98. Memória, contagem de referências e o coletor de lixo

(em elaboracao)

Parte XVI.

**Volume 16 — Fronteira e Prática
Contínua**

99. Estendendo Python: C, Cython e ligações nativas

(em elaboracao)

100. Python acelerado: PyPy, Numba e o interpretador sem GIL

(em elaboracao)

101. Arquitetura de projetos e código sustentável

(em elaboracao)

102. Git, pre-commit e integração contínua para projetos Python

(em elaboracao)

103. Python e inteligência artificial: LLMs, APIs e agentes

(em elaboracao)

104. Para onde ir depois: PEPs, comunidade e prática contínua

(em elaboracao)

Referências

